



Desarrollo de un sistema RAG (Retrieval Augmented Generation) mediante el uso de LLMs

Autor

José Manuel Román Rodríguez

Director

Juan Francisco Huete Guadix



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

—
Granada, junio de 2026

Índice general

Resumen	7
Abstract	9
1. Introducción	17
1.1. Descripción del problema	17
1.2. Motivación	19
1.3. Objetivos	20
1.4. Planificación	21
2. Requisitos	25
2.1. OpenSearch	25
2.1.1. Prueba OpenSearch	26
2.2. Requisitos para poder ejecutar modelos de lenguaje	27
2.2.1. Instalación drivers Nvidia	28
2.2.2. Instalación de python y librerías necesarias	28
2.2.3. Ejemplo de prueba GPT-2	29
3. Fundamentos teóricos	33
3.1. Best Matching 25	33
3.1.1. Formulación de BM25	34
3.1.2. Componentes del modelo	34
3.1.3. Justificación matemática (Intuición probabilística)	36
3.2. Embeddings	36
3.2.1. Tipos de embeddings	37
3.2.2. Similitud y uso en recuperación	38
3.2.3. Ventajas, limitaciones y usos en RAG	40
3.3. Modelos largos de lenguaje	41
3.3.1. Arquitectura base: Transformers	41
3.3.2. Proceso de entrenamiento y capacidades	44
3.3.3. Limitaciones y uso en RAG	45

4. Estado del arte	47
4.1. Introducción a RAG	47
4.2. Sistemas clásicos de recuperación neural	48
4.3. Evolución de RAG	49
4.4. Mejoras recientes y aplicaciones reales	51
4.5. Comparación con el trabajo realizado	52
5. Materiales y métodos	55
5.1. Conjunto de datos	55
5.2. Preprocesamiento e indexación	56
5.2.1. Extracción y estructuración de documentos	56
5.2.2. Fragmentación en chunks	57
5.2.3. Indexación semántica mediante embeddings	58
5.2.4. Pseudocódigo del proceso de indexación	60
5.3. Métricas	60
5.3.1. Métricas de recuperación	61
5.3.2. Métricas de evaluación de generación	61
5.4. Configuración experimental	62
5.4.1. Entorno de ejecución	62
5.4.2. Modelo generativo y sistema de recuperación	62
5.4.3. Dataset de evaluación	65
6. Implementación	69
6.1. Componentes comunes	69
6.1.1. Carga del Dataset y conexión con el motor de búsqueda	70
6.1.2. Carga del modelo y cuantización	71
6.1.3. Generación con el LLM	73
6.1.4. Evaluación el sistema	75
6.2. Sistemas de recuperación	79
6.2.1. BM25	80
6.2.2. Embeddings	82
6.2.3. Híbridos	83
6.3. Uso del sistema	87
7. Evaluación experimental y resultados	89
7.1. Experimentación con en número de chunks recuperados . . .	90
7.2. Resultados de métricas de retrieval	96
7.3. Experimentación con distintos valores de la temperatura del modelo generativo	98
7.4. Comparación de respuestas: RAG frente a LLM	101
7.5. Experimentación con LLM-as-a-Judge	103
7.5.1. Auto-evaluación con el modelo generador	104
7.5.2. Evaluación con un modelo juez externo	106

ÍNDICE GENERAL	5
8. Conclusiones y trabajos futuros	109
Bibliografía	117

Resumen

En los últimos años, el desarrollo de modelos de lenguaje de gran tamaño (LLMs, Large Language Models) ha supuesto un avance significativo en el ámbito del procesamiento del lenguaje natural. Sin embargo, estos modelos presentan limitaciones importantes, como la generación de información incorrecta o desactualizada y la falta de acceso directo a conocimiento específico o privado. En este contexto, surgen los sistemas de Retrieval-Augmented Generation (RAG), que combinan técnicas de recuperación de información con modelos generativos para mejorar la calidad, precisión y trazabilidad de las respuestas.

El presente Trabajo de Fin de Grado aborda el diseño e implementación de un sistema RAG basado en LLMs, centrado en la integración de diferentes estrategias de recuperación de información. En particular, se han empleado técnicas basadas en representaciones vectoriales mediante embeddings, así como métodos clásicos de recuperación como BM25. Además, se han desarrollado enfoques híbridos que combinan ambas aproximaciones con el objetivo de aprovechar las ventajas de cada una: la capacidad semántica de los embeddings y la eficacia léxica de BM25.

Un aspecto clave del proyecto ha sido la definición e implementación de métricas para evaluar la calidad de los resultados obtenidos. Estas métricas permiten analizar tanto la relevancia de los documentos recuperados como la calidad final de las respuestas generadas, proporcionando un marco objetivo para comparar las distintas técnicas empleadas.

Como resultado, se ha desarrollado un sistema funcional que demuestra cómo la combinación de diferentes métodos de recuperación mejora significativamente el rendimiento frente a soluciones basadas en una única técnica.

En conclusión, este trabajo pone de manifiesto el potencial de los sistemas RAG como herramienta para mejorar las capacidades de los LLMs, destacando la importancia de combinar distintas técnicas de recuperación y de contar con métricas adecuadas que permitan evaluar su rendimiento de forma rigurosa.

Abstract

In recent years, the development of Large Language Models (LLMs) has represented a significant advancement in the field of Natural Language Processing. However, these models present important limitations, such as the generation of incorrect or outdated information and the lack of direct access to specific or private knowledge. In this context, Retrieval-Augmented Generation (RAG) systems have emerged as a solution that combines information retrieval techniques with generative models to improve the quality, accuracy, and traceability of responses.

This Bachelor's Thesis focuses on the design and implementation of an RAG system based on LLMs, with an emphasis on the integration of different information retrieval strategies. In particular, embedding-based techniques using vector representations have been employed, along with classical retrieval methods such as BM25. Additionally, hybrid approaches have been developed to combine both methods, aiming to leverage their respective strengths: the semantic capabilities of embeddings and the lexical effectiveness of BM25.

A key aspect of this project has been the definition and implementation of metrics to evaluate the quality of the obtained results. These metrics allow for the assessment of both the relevance of the retrieved documents and the overall quality of the generated responses, providing an objective framework to compare the different techniques used.

As a result, a functional system has been developed that demonstrates how the combination of different retrieval methods significantly improves performance compared to solutions based on a single technique.

In conclusion, this work highlights the potential of RAG systems as a tool to enhance the capabilities of LLMs, emphasizing the importance of combining multiple retrieval techniques and employing appropriate evaluation metrics to rigorously assess their performance.

Yo, **José Manuel Román Rodríguez**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI **49629771A**, declaro la originalidad de este trabajo y que **todas las fuentes utilizadas han sido citadas adecuadamente** y autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: José Manuel Román Rodríguez

Granada a 21 de junio de 2026 .

D. **Juan Francisco Huete Guadix**, Profesor del Área de Ciencias de la Computación e Inteligencia Artificial del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Desarrollo de un sistema RAG (Retrieval Augmented Generation) mediante el uso de LLMs***, ha sido realizado bajo su supervisión por **José Manuel Román Rodríguez**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a 21 de junio de 2026 .

Los directores:

Juan Francisco Huete Guadix

Agradecimientos

En primer lugar, quiero expresar mi más sincero agradecimiento a toda mi familia al completo por su apoyo incondicional a lo largo de todos estos años. Su comprensión, paciencia y ánimo constante han sido fundamentales para poder afrontar este proceso con motivación y perseverancia.

Asimismo, deseo mostrar mi agradecimiento a mi tutor por su orientación, dedicación y seguimiento continuo del trabajo. Sus consejos, observaciones y apoyo académico han sido esenciales para la correcta realización de este proyecto.

Capítulo 1

Introducción

1.1. Descripción del problema

En la última década, el campo del procesamiento del lenguaje natural ha experimentado un crecimiento exponencial impulsado por el desarrollo de modelos de lenguaje de gran tamaño (Large Language Models, LLMs) [20, 30, 4]. Estos modelos, entrenados sobre grandes volúmenes de datos textuales, han demostrado una notable capacidad para generar texto coherente, responder preguntas y asistir en tareas complejas, lo que ha propiciado su adopción en múltiples aplicaciones, desde asistentes virtuales hasta herramientas de apoyo a la toma de decisiones. Este avance ha estado estrechamente ligado al incremento de la capacidad computacional y a la disponibilidad de grandes conjuntos de datos, lo que ha permitido entrenar modelos cada vez más complejos y precisos.

A pesar de sus avances, los LLMs presentan limitaciones significativas. Entre ellas destacan la generación de respuestas incorrectas o no verificables, conocidas como alucinaciones [20], así como su incapacidad para acceder a información actualizada o a fuentes de conocimiento específicas no incluidas en su entrenamiento. Estas limitaciones suponen un desafío importante en contextos donde la precisión y la fiabilidad de la información son críticas, como en aplicaciones médicas, legales o empresariales. Además, la naturaleza estática de los datos de entrenamiento implica que el conocimiento del modelo puede quedar obsoleto con el paso del tiempo, reduciendo su utilidad en escenarios dinámicos.

Para abordar estos problemas, han surgido los sistemas de Retrieval-Augmented Generation (RAG) [20, 24], que combinan técnicas de recuperación de información con modelos generativos. En lugar de depender únicamente del conocimiento interno del modelo, los sistemas RAG incorporan información externa relevante en tiempo de consulta, lo que permite generar respuestas más precisas, actualizadas y fundamentadas en fuentes concretas. Este enfoque no solo mejora la calidad de las respuestas, sino que también

aporta mayor transparencia, al poder vincular la información generada con documentos específicos.

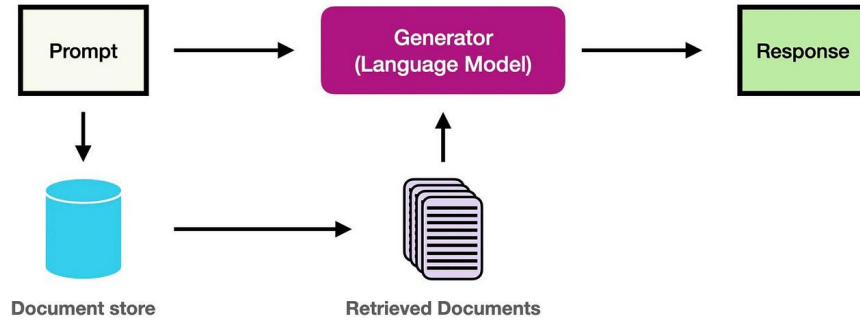


Figura 1.1: Arquitectura general de un sistema RAG. Fuente: [31]

El funcionamiento de estos sistemas se basa en varias etapas clave (Figura 1.1). En primer lugar, se realiza una fase de indexación de documentos, en la que la información se organiza y transforma en representaciones adecuadas para su posterior recuperación. Esta fase puede incluir procesos de limpieza, segmentación y transformación de los documentos. A continuación, ante una consulta del usuario, el sistema recupera los documentos más relevantes mediante distintas técnicas, que pueden incluir métodos clásicos como BM25 [47] o enfoques semánticos basados en embeddings [20, 21] (Figura 1.2). Finalmente, esta información se integra en el contexto de un modelo generativo, que produce una respuesta enriquecida. La correcta interacción entre estas etapas resulta clave para garantizar el rendimiento global del sistema.

En este contexto, el presente trabajo se centra en el estudio, diseño e implementación de un sistema RAG que combina diferentes estrategias de recuperación de información. Se presta especial atención a la integración de métodos basados en embeddings, técnicas clásicas como BM25 y enfoques híbridos [25], con el objetivo de analizar sus ventajas y limitaciones. La combinación de estos métodos permite equilibrar la precisión léxica y la comprensión semántica, mejorando la capacidad del sistema para recuperar información relevante en distintos tipos de consultas.

Asimismo, se aborda la definición de métricas [48] que permitan evaluar de forma objetiva la calidad de los resultados obtenidos, tanto en términos de relevancia de los documentos recuperados como de la calidad de las respuestas generadas. La evaluación constituye un aspecto fundamental en este tipo de sistemas, ya que permite comparar distintas configuraciones y técnicas, así como identificar posibles áreas de mejora.

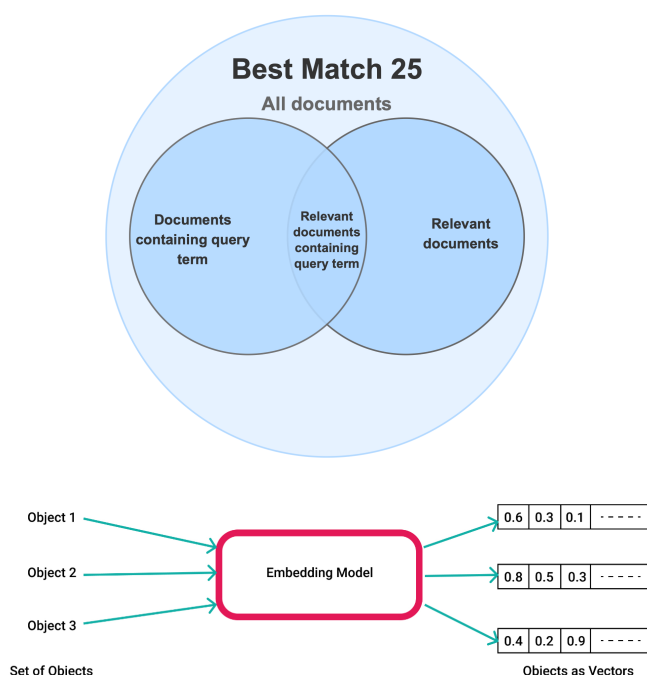


Figura 1.2: Comparación entre recuperación léxica y semántica. Fuentes: [44][1]

Este enfoque permite no solo comprender en profundidad el funcionamiento de los sistemas RAG, sino también analizar cómo la combinación de distintas técnicas puede mejorar significativamente su rendimiento, sentando las bases para futuras aplicaciones en entornos donde la precisión y el acceso a información actualizada resultan fundamentales. En este sentido, el trabajo contribuye a evidenciar la importancia de integrar múltiples estrategias de recuperación y de establecer metodologías de evaluación rigurosas en el desarrollo de sistemas basados en LLMs.

1.2. Motivación

En el contexto de los modelos de lenguaje de gran tamaño, el desarrollo de técnicas de Retrieval-Augmented Generation (RAG) [24, 20] ha permitido mejorar significativamente la calidad de las respuestas al incorporar información externa durante el proceso de generación. Sin embargo, en muchas implementaciones de estos sistemas, el foco principal suele centrarse en optimizar la calidad de las respuestas generadas o la precisión del componente

de recuperación, sin analizar en profundidad cómo las distintas decisiones de diseño afectan al comportamiento global del sistema.

A continuación, se presentan los aspectos clave que motivan este trabajo:

- **Variabilidad en las implementaciones de RAG:** Las arquitecturas basadas en RAG pueden implementarse de múltiples formas, variando elementos como la segmentación de documentos (chunking), el método de indexación, el modelo de recuperación o la forma en la que se integra la información recuperada con el modelo generativo. Estas decisiones influyen de manera significativa en el rendimiento final del sistema, lo que hace necesario un análisis comparativo de distintas configuraciones.
- **Impacto en el rendimiento global del sistema:** Pequeñas modificaciones en cualquiera de las etapas del sistema RAG pueden afectar no solo a la relevancia de los documentos recuperados, sino también a la coherencia, precisión y utilidad de las respuestas generadas. Por ello, resulta relevante estudiar de forma sistemática cómo estas variaciones afectan al rendimiento global.
- **Complejidad en la evaluación de sistemas RAG:** La evaluación de estos sistemas no es trivial, ya que no existe una única métrica que capture todos los aspectos relevantes del rendimiento. Es necesario considerar simultáneamente la calidad de la respuesta generada, la pertinencia de la información recuperada y la coherencia entre ambos componentes, lo que implica el uso de métricas de evaluación específicas y complementarias [48].
- **Necesidad de un análisis comparativo:** El estudio conjunto de distintas implementaciones y métricas de evaluación permite obtener una visión más completa del comportamiento de los sistemas RAG. Esto facilita la identificación de ventajas y limitaciones de cada enfoque, contribuyendo al diseño de sistemas más robustos y adecuados a distintos contextos de aplicación.

1.3. Objetivos

Se parte de la hipótesis inicial de que las distintas decisiones de diseño en los sistemas de Retrieval-Augmented Generation (RAG), tales como el método de segmentación de documentos, el modelo de recuperación o la estrategia de integración entre recuperación y generación, pueden afectar de manera significativa al rendimiento global del sistema. Asimismo, se considera que la evaluación de estos sistemas no puede basarse únicamente en métricas de calidad de generación, sino que debe incorporar también el análisis de la calidad de la información recuperada.

En este contexto, este trabajo tiene como objetivo principal analizar y comparar distintas estrategias de implementación de sistemas RAG, así como estudiar las métricas utilizadas para su evaluación, con el fin de comprender su impacto en el rendimiento global del sistema y sus posibles limitaciones.

A tal efecto, este objetivo principal se descompone en varios objetivos parciales:

- Estudiar diferentes estrategias de implementación de sistemas RAG, considerando variaciones en aspectos como la segmentación de documentos (chunking), los métodos de indexación y el uso de embeddings [47, 21, 25].
- Comparar el comportamiento de distintas configuraciones de sistemas RAG bajo un marco de evaluación común, analizando su impacto en la calidad de las respuestas generadas.
- Investigar y analizar las principales métricas utilizadas en la evaluación de sistemas RAG, evaluando su capacidad para reflejar tanto la calidad de la generación como la relevancia de la información recuperada [48].
- Identificar posibles limitaciones en las estrategias de implementación y en las métricas de evaluación, proponiendo líneas de mejora para futuros trabajos.

1.4. Planificación

Este trabajo corresponde a un TFG de 12 créditos ECTS (European Credit Transfer and Accumulation System), que equivalen aproximadamente a unas 300 horas de trabajo. Teniendo en cuenta que la planificación está prevista para el segundo cuatrimestre del curso, comprendido entre los meses de enero y junio, se consta con 6 meses para su finalización. Reservando la última semana en caso de posibles retrasos e imprevistos, el tiempo total resultaba en 23 semanas de desarrollo, lo que se traduce en aproximadamente unas 14 horas de trabajo semanales. A continuación, se detalla el ciclo de vida del proyecto.

A la hora de realizar las distintas fases del proyecto se utilizó un enfoque basado en una metodología de carácter iterativo e incremental [23, 49] en el que el sistema se construye progresivamente y se evalúa de forma continua. A continuación se definen las fases del ciclo de vida del proyecto:

- **Fase de definición del problema y revisión bibliográfica**
 - Reuniones iniciales con el tutor para delimitar el problema de trabajo.

- Definición del enfoque general basado en sistemas de Recuperación Aumentada por Generación (RAG).
- Revisión del estado del arte y estudio de la bibliografía relevante.

■ **Fase de diseño conceptual del sistema**

- Diseño de la arquitectura general del pipeline RAG.
- Toma de decisiones sobre los componentes principales del sistema.
- Selección de OpenSearch como motor de indexación y recuperación.
- Definición de los modelos de lenguaje y enfoques de recuperación a utilizar.

■ **Fase de implementación incremental del sistema**

- Implementación del proceso de indexación del corpus documental.
- Desarrollo del sistema base de recuperación utilizando BM25 como baseline.
- Implementación de la recuperación basada en embeddings y creación del índice vectorial.
- Desarrollo de los distintos sistemas híbridos de recuperación, incluyendo integración en OpenSearch y estrategias basadas en Reciprocal Rank Fusion (RRF).

■ **Fase de evaluación experimental**

- Diseño e implementación de métricas de evaluación para retrieval y generación.
- Evaluación comparativa de los distintos sistemas de recuperación.
- Análisis de resultados y comparación entre enfoques BM25, embeddings e híbridos.

■ **Fase de documentación**

- Redacción progresiva de la memoria del trabajo.
- Integración de resultados experimentales y descripción del sistema.

Fases	Semanas	Horas	Enero				Febrero				Marzo				Abril				Mayo				Junio			
			S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4	S1	S2	S3	S4
Fase de definición del problema	1	14																								
Fase de diseño	1	14																								
Fase de implementación	11	154																								
Fase de evaluación	2	28																								
Fase de documentación y revisión	8	112																								

Figura 1.3: Planificación del proyecto. En rojo la última semana reservada a imprevistos y revisión.

La Figura 1.3 muestra un diagrama de Gantt donde se muestra toda la planificación. En él, se muestra la estructuración de las 23 semanas para cada una de las fases, reservando la última semana para imprevistos y revisión del trabajo realizado.

Como puede observarse la fase que más tiempo llevó es la fase de implementación ya que se incluye en ella parte de la experimentación realizada, esto hizo que el tiempo se elevara considerablemente —teniendo en cuenta los posibles tiempos de espera derivados de la ejecución durante la experimentación—, además de problemas derivados en la implementación del primer RAG (BM25) que fue el primero y más complicado.

A la hora de calcular los costes del proyecto (Figura 1.4), se establece una retribución de 10 euros la hora para un investigador junior, acorde a los salarios habituales en universidades o centros públicos. Además, se contemplan los gastos asociados a los materiales y equipamiento necesarios para la realización del proyecto, entre ellos un equipo portátil de gama media-baja con una tarjeta gráfica de gama media-baja, dedicado tanto a la experimentación realizado como a la redacción de la memoria.

Item	Precio unidad	Unidades	Subtotal
Horas trabajo investigador junior	10,00 €	322	3.220,00 €
Equipo portátil gama media-baja	900,00 €	1	900,00 €
Total			4.120,00 €

Figura 1.4: Costes asociados al proyecto.

Capítulo 2

Requisitos

En este capítulo se describen los requisitos necesarios para la correcta puesta en marcha del sistema desarrollado, así como el entorno de ejecución utilizado durante su implementación y evaluación. En particular, se detallan tanto los componentes software como las dependencias hardware necesarias para garantizar el funcionamiento de las distintas partes del sistema, incluyendo el motor de búsqueda basado en OpenSearch [36] y los modelos de lenguaje utilizados en el procesamiento de consultas.

Además, se presentan los pasos de instalación y verificación de los principales componentes, con el objetivo de asegurar una configuración reproducible del entorno. Se incluyen pruebas básicas de funcionamiento tanto de OpenSearch como de la infraestructura necesaria para la ejecución de modelos basados en redes neuronales, haciendo especial énfasis en la configuración de GPU mediante CUDA [33] y el uso de librerías como PyTorch [41] y Transformers [51]. De este modo, se proporciona una base sólida para el desarrollo y ejecución del sistema completo.

2.1. OpenSearh

En esta sección se detalla el proceso de instalación y verificación de OpenSearch en un entorno Windows, así como las pruebas básicas necesarias para asegurar su correcto funcionamiento. En primer lugar, se describe la descarga y extracción del paquete oficial, teniendo en cuenta ciertas consideraciones importantes en la ruta de instalación para evitar posibles errores.

Posteriormente, se presentan los pasos para la comprobación del servicio, tanto con la configuración de seguridad habilitada, como mediante la ejecución del servicio desde línea de comandos. Finalmente, se incluyen varias pruebas de conectividad mediante peticiones HTTP al clúster, lo que permite validar que OpenSearch está operativo y que sus plugins principales han sido correctamente cargados.

Comenzamos descargando el archivo *opensearch-3.3.2-windows-x64.zip*

de la página oficial de OpenSearch (<https://docs.opensearch.org/latest/install-and-configure/install-opensearch/windows/>).

Continuamos extrayendo el contenido del archivo, asegurándose que no haya espacios en la ruta de extracción.

2.1.1. Prueba OpenSearch

Este paso es importante ya que de lo contrario si aparecen problemas en el futuro puede ser complicado determinarlos.

Para comprobar que la instalación es correcta lo podemos hacer de dos maneras, con la seguridad habilitada y con la seguridad deshabilitada.

Procedemos a explicar con la seguridad habilitada. Para ello ejecutamos el script de lote de demostración desde el símbolo del sistema.

Abrimos el símbolo del sistema y cambiamos al directorio donde está la instalación de OpenSearch.

```
1 cd /path/to/opensearch-3.3.2
```

Ahora debemos usar el siguiente comando para especificar una contraseña de administrador.

```
1 set OPENSEARCH\_INITIAL\_ADMIN\_PASSWORD=<custom-admin-  
password>
```

```
1 .\opensearch-windows-install.bat
```

A continuación debemos abrir otro símbolo del sistema y enviamos solicitudes al servidor para verificar que está funcionando correctamente.

```
1 curl.exe -X GET https://localhost:9200 -u  
2 "admin:<custom-admin-password>" --insecure
```

Importante el parámetro `--insecure`, ya que es necesario porque los certificados TLS son autofirmados.

Con esta orden enviamos una solicitud al puerto 9200

Se debería recibir una respuesta parecida a esta:

```
1 {  
2   "name" : "hostname-here",  
3   "cluster_name" : "opensearch",  
4   "cluster_uuid" : "7Nqtr0LrQT0veFcBb7Kufw",  
5   "version" : {  
6     "distribution" : "opensearch",  
7     "number" : <version>,
```

```

8     "build_type" : <build-type>,
9     "build_hash" : <build-hash>,
10    "build_date" : <build-date>,
11    "build_snapshot" : false,
12    "lucene_version" : <lucene-version>,
13    "minimum_wire_compatibility_version" : "7.10.0",
14    "minimum_index_compatibility_version" : "7.0.0"
15  },
16  "tagline" : "The OpenSearch Project: https://
    opensearch.org/"
17 }

```

Por último consultamos el punto final de los complementos

```

1 curl.exe -X GET https://localhost:9200/_cat/plugins?v -u
2 "admin:<custom-admin-password>" --insecure

```

La respuesta debería verse de la siguiente forma:

```

1 hostname opensearch-alerting 3.3.2
2 hostname opensearch-anomaly-detection 3.3.2
3 hostname opensearch-asynchronous-search 3.3.2
4 hostname opensearch-cross-cluster-replication 3.3.2
5 hostname opensearch-geospatial 3.3.2
6 hostname opensearch-index-management 3.3.2
7 hostname opensearch-job-scheduler 3.3.2
8 hostname opensearch-knn 3.3.2
9 hostname opensearch-ml 3.3.2
10 hostname opensearch-neural-search 3.3.2
11 hostname opensearch-notifications 3.3.2
12 hostname opensearch-notifications-core 3.3.2
13 hostname opensearch-observability 3.3.2
14 hostname opensearch-reports-scheduler 3.3.2
15 hostname opensearch-security 3.3.2
16 hostname opensearch-security-analytics 3.3.2
17 hostname opensearch-sql 3.3.2

```

2.2. Requisitos para poder ejecutar modelos de lenguaje

En esta sección se describen los requisitos necesarios para la ejecución de modelos de lenguaje en el entorno utilizado en este trabajo, incluyendo tanto los componentes hardware como las dependencias software necesarias para aprovechar la aceleración mediante GPU. En particular, se detalla el proceso de instalación y configuración de los drivers de NVIDIA [34], la instalación de Python y de las librerías principales como PyTorch y Transformers, así

como las comprobaciones necesarias para asegurar que el entorno está correctamente preparado. Finalmente, se incluye un ejemplo práctico con el modelo GPT-2 [42] con el objetivo de validar el funcionamiento de CUDA y confirmar que es posible ejecutar modelos de lenguaje de forma eficiente en la GPU.

2.2.1. Instalación drivers Nvidia

Primero debemos instalar los drivers adecuados para la tarjeta gráfica que tengamos, para ello entramos en la página oficial de Nvidia (<https://www.nvidia.com/es-la/drivers/>) y buscamos el nombre de la tarjeta que tengamos.

Bastaría con poner por ejemplo "2060". Y con esto saldría automáticamente una lista de drivers, ahí elegimos los que queremos descargar.

Comprobamos que todo está correcto ejecutando la siguiente orden en un símbolo del sistema.

```
1 nvidia-smi
```

Debería aparecer información de la tarjeta gráfica.

```
1 NVIDIA GeForce RTX 2060
2 Driver Version: 5xx.xx
3 CUDA Version: 12.x
```

2.2.2. Instalación de python y librerías necesarias

Comprobamos la versión de python que tenemos en caso de tenerlo instalado ya con la siguiente orden.

```
1 python --version
```

Debería aparecer algo como esto:

```
1 python 3.x.x
```

Es importante tener una versión inferior a la 3.12, ya que hay problemas con Pytorch y con algunas dependencias.

En el caso de que no esté instalado python debemos ir a la página oficial de python (<https://www.python.org/downloads/windows/>) y descargar el instalador de la versión 3.11.

Una vez descargado el instalador lo ejecutamos y seguimos los pasos de instalación.

A continuación debemos instalar Pytorch con soporte de CUDA para poder utilizar la tarjeta gráfica, lo hacemos ejecutando la siguiente orden.

```
1 pip install torch --index-url
2 https://download.pytorch.org/whl/cu118
```

Además también debemos instalar Transformers, ya que es necesario porque es la librería que contiene los modelos como GPT-2, para ello ejecutamos la siguiente orden.

```
1 pip install transformers
```

2.2.3. Ejemplo de prueba GPT-2

Creamos un archivo .py y comprobamos con CUDA que la tarjeta gráfica está funcionando.

```
1 import torch
2
3 print("CUDA disponible:", torch.cuda.is_available())
4 if torch.cuda.is_available():
5     print("GPU:", torch.cuda.get_device_name(0))
```

Este fragmento comprueba si la GPU está disponible para Pytorch y si lo está muestra su nombre. En la primera línea se devuelve True, si Pytorch detecta una GPU NVIDIA con soporte en CUDA y False si no detecta nada.

En el caso de que sea True se obtiene esta respuesta.

```
1 CUDA disponible: True
2 GPU: NVIDIA GeForce RTX 2060
```

Continuamos con el uso del modelo GPT-2 [42], para ello visitamos la página Hugging Face (<https://huggingface.co/>), ya que es la forma más fácil y cómoda de usar modelos, en ella basta con buscar el modelo que queremos usar, GPT-2, y nos proporciona una guía de como usarlo.

```
1 from transformers import pipeline, set_seed
2
3 device = 0 if torch.cuda.is_available() else -1
4 generator = pipeline('text-generation', model='gpt2',
5                       device=device)
6 set_seed(42)
7 resultados = generator("Hello, I'm a language model,",
8                        max_new_tokens=50,
9                        num_return_sequences=5,
10                       truncation=True)
11 for r in resultados:
```

```
12 print("\n==>", r["generated_text"])
```

Este es el código de ejemplo proporcionado por Hugging Face, primero se importan dos cosas, **pipeline**, una herramienta que facilita usar modelos ya configurados y **set_seed**, que sirve para fijar una semilla aleatoria, para que se repitan los resultados.

Después se elige entre CPU o GPU, dependiendo de lo que devuelva CUDA.

Se crea el generador basado en GPT-2

```
1 generator = pipeline('text-generation', model='gpt2',
2                       device= device)
```

Aquí se está creando un generador de texto (text-generation), que usa el modelo GPT-2(gpt2), con GPU o CPU dependiendo de antes (device).

Internamente hace:

- Descargar GPT-2
- Cargar sus pesos
- Cargar el tokenizador BPE
- Preparar el modelo para Pytorch
- Enviarlo a GPU si se puede

Por último se genera el texto.

```
1 resultados = generator(
2     "Hello, I'm a language model,",
3     max_new_tokens=50,
4     num_return_sequences=5,
5     truncation=True
6 )
```

Los parámetros aquí son los siguientes:

- "Hello, I'm a language model", este es el prompt
- max_new_token=50, GPT-2 generará 50 tokens adicionales.
- num_return_sequences=5, 5 textos diferentes con la misma entrada.
- truncation=True, si la respuesta es muy larga se corta.

Se muestran los textos y esta sería una posible respuesta.

CUDA disponible: True

GPU: NVIDIA GeForce RTX 2060

Device set to use cuda:0

Setting 'pad_token_id' to 'eos_token_id':50256 for open-end generation.

=== Hello, I'm a language model, so that's a hard one, but let's see how we can achieve it!

You can learn more about this topic in this talk — a lot of that is code, but there's also a lot of code that goes into how to

=== Hello, I'm a language model, not an ontology. I don't want to write an ontology of any sort. I want a language that can read a lot of data and understand what it's doing. So, I'm not saying that you can't write a language with === Hello, I'm a language model, but I don't have a whole lot of experience with languages and I don't know much about them. I'm really curious about what they're like, what they're like in general, and what they do differently. I think they all have their

Capítulo 3

Fundamentos teóricos

En este capítulo se presenta el marco teórico necesario para la correcta comprensión de los sistemas de generación aumentada por recuperación (RAG)[24] y de las distintas técnicas empleadas en su implementación. Asimismo, se establecen los fundamentos que permiten interpretar adecuadamente los experimentos realizados y analizar de forma crítica los resultados obtenidos en capítulos posteriores.

Para ello, se abordan en primer lugar los métodos clásicos de recuperación de información, como BM25 [47], que constituyen la base de muchos sistemas de búsqueda. A continuación, se introducen los embeddings [21], fundamentales para capturar similitud semántica en representaciones vectoriales densas.

Posteriormente, se presentan los modelos de lenguaje de gran tamaño (LLMs) [20, 4, 30], que permiten la generación de texto coherente y contextualizado. Finalmente, se describe la arquitectura Retrieval-Augmented Generation, que integra recuperación y generación para mejorar la precisión y actualidad de las respuestas.

3.1. Best Matching 25

En el ámbito de la recuperación de información, uno de los objetivos principales es determinar la relevancia de un documento dado con respecto a una consulta formulada por el usuario. Tradicionalmente, este problema se ha abordado mediante modelos basados en el análisis estadístico de los términos, donde la frecuencia de aparición de las palabras juega un papel fundamental. En este tipo de enfoques, conocidos como modelos de bolsa de palabras (*bag-of-words*) [20, 27], los documentos y las consultas se representan como conjuntos no ordenados de términos, ignorando la estructura y el contexto lingüístico.

En este contexto, Best Matching 25 (BM25) [47] se ha consolidado como uno de los modelos de ranking más utilizados debido a su eficacia y sim-

plicidad. BM25 pertenece a la familia de modelos probabilísticos y se basa en la idea de que la relevancia de un documento puede estimarse a partir de la frecuencia de los términos de la consulta en el documento, ajustada por la longitud del mismo y la frecuencia inversa de dichos términos en la colección. Como modelo basado en bolsa de palabras, BM25 no considera el orden ni las relaciones semánticas entre los términos, sino únicamente su presencia y distribución.

Este modelo introduce mejoras respecto a enfoques anteriores como TF-IDF [19], incorporando mecanismos de normalización que evitan que documentos más largos sean favorecidos injustamente y controlando la influencia de términos muy frecuentes.

3.1.1. Formulación de BM25

BM25 asigna una puntuación de relevancia a un documento d dado una consulta q , basada en la suma de contribuciones de cada término de la consulta. Su formulación general es [47]:

$$score(d, q) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}$$

donde:

- t es un término de la consulta.
- $f(t, d)$ es la frecuencia del término t en el documento d .
- $|d|$ es la longitud del documento (en número de términos).
- $avgdl$ es la longitud media de los documentos de la colección.
- k_1 y b son parámetros del modelo.

3.1.2. Componentes del modelo

Frecuencia del término (TF)

El término $f(t, d)$ mide cuántas veces aparece un término en un documento. A diferencia de TF-IDF [19], en BM25 esta frecuencia no crece linealmente, sino que está controlada por el denominador, cuando $f(t, d)$ es pequeño, el impacto crece rápidamente y cuando es grande, el crecimiento se satura.

Esto evita que repetir muchas veces una palabra distorsione la relevancia.

Frecuencia inversa del término (IDF)

El peso de cada término viene dado por:

$$IDF(t) = \log \left(\frac{N - n(t) + 0.5}{n(t) + 0.5} \right)$$

donde:

- N es el número total de documentos.
- $n(t)$ es el número de documentos que contienen el término.

De este modo, a los términos muy raros se les da un mayor peso, y a términos muy frecuentes tienen un mayor peso.

Normalización por longitud

El término:

$$\frac{|d|}{avgdl}$$

Esto permite ajustar la puntuación según la longitud del documento, para que en documentos largos no se vean favorecidos artificialmente y en documentos cortos no se penalicen en exceso.

El parámetro b controla este efecto:

- $b = 1$: normalización completa.
- $b = 0$: sin normalización.

Parámetro k_1

Controla la influencia de la frecuencia del término:

- valores típicos: $k_1 \in [1.2, 2.0]$
- valores altos $\rightarrow$ más peso a la frecuencia.
- valores bajos $\rightarrow$ saturación más rápida.

Vemos que la fórmula puede entenderse como una combinación de tres factores:

- **Importancia del término en la colección $\rightarrow$ IDF**
- **Importancia en el documento $\rightarrow f(t, d)$**
- **Corrección estructural $\rightarrow$ longitud del documento**

Adicionalmente, en la formulación general de BM25 también se considera la **frecuencia del término en la consulta**, denotada como $f(t, q)$, lo que permite modelar la importancia relativa de cada término dentro de la propia consulta.

De este modo, la expresión completa de BM25 puede escribirse como:

$$\text{score}(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \cdot \frac{f(t, q) (k_3 + 1)}{f(t, q) + k_3}$$

donde:

- k_3 es un parámetro que controla la influencia de la frecuencia del término en la consulta.

3.1.3. Justificación matemática (Intuición probabilística)

El desarrollo formal del modelo BM25 puede encontrarse en la literatura original sobre el Binary Independence Model y en su posterior formulación como BM25 [46, 47], donde se detallan las aproximaciones probabilísticas y las modificaciones empíricas que dan lugar a su expresión final utilizada en sistemas de recuperación de información modernos.

3.2. Embeddings

En el ámbito del procesamiento del lenguaje natural, uno de los principales retos consiste en representar el texto de manera que pueda ser procesado eficazmente por algoritmos computacionales. Tradicionalmente, los métodos de representación se basaban en enfoques discretos, donde cada palabra o término se trataba como una unidad independiente sin tener en cuenta su significado o contexto. Sin embargo, estos enfoques presentan limitaciones importantes, especialmente a la hora de capturar relaciones semánticas entre palabras. Este tipo de representación da lugar a sistemas de búsqueda basados en términos, en los que la relevancia se determina principalmente por la coincidencia exacta de palabras entre la consulta y los documentos.

En este contexto, los **embeddings** [21] surgen como una solución basada en representaciones vectoriales continuas. Un embedding es una función que transforma unidades lingüísticas, como palabras, frases o documentos, en vectores numéricos dentro de un espacio de alta dimensión. En dicho espacio vectorial, la distancia o similitud entre vectores refleja relaciones semánticas entre los elementos representados, de modo que palabras con significados similares tienden a situarse próximas entre sí, mientras que aquellas con significados distintos se encuentran más alejadas. Este enfoque permite realizar búsqueda semántica, en la que los documentos se recuperan en función de su significado, incluso cuando no comparten términos exactos con la consulta.

A diferencia de enfoques clásicos como TF-IDF [19] o BM25 [47], que dependen de la coincidencia exacta de términos, los embeddings permiten capturar similitudes semánticas incluso cuando no existe solapamiento léxico. Esto supone un avance significativo en tareas donde el significado es más importante que la forma superficial del texto, como la búsqueda semántica o la recuperación de información basada en contexto.

El desarrollo de embeddings ha evolucionado desde modelos estáticos de palabras, como Word2Vec [29] o GloVe [39], hasta modelos más avanzados basados en arquitecturas profundas, como BERT [10], que generan representaciones contextuales en función del entorno lingüístico. En la actualidad, los embeddings constituyen un componente fundamental en numerosos sistemas de inteligencia artificial, especialmente en aquellos relacionados con la recuperación de información y la generación de lenguaje natural, desempeñando un papel clave en arquitecturas como Retrieval-Augmented Generation [24].

3.2.1. Tipos de embeddings

Los embeddings han evolucionado significativamente desde sus primeras formulaciones hasta los modelos actuales, dando lugar a diferentes tipos de representaciones en función de cómo capturan la información semántica y contextual del lenguaje.

En primer lugar, se encuentran los embeddings estáticos, en los que cada palabra se representa mediante un único vector fijo, independientemente del contexto en el que aparezca. Entre los modelos más representativos destacan Word2Vec (Figura 3.1) [29] y GloVe [39]. Estos enfoques permiten capturar relaciones semánticas generales entre palabras a partir de su coocurrencia en grandes corpus de texto. Sin embargo, presentan la limitación de no poder distinguir entre diferentes significados de una misma palabra, ya que asignan siempre la misma representación vectorial.

Para abordar esta limitación, surgen los embeddings contextuales, que generan representaciones dinámicas en función del contexto lingüístico. Un ejemplo destacado es BERT [10], que utiliza arquitecturas basadas en transformadores para producir vectores que dependen de las palabras que rodean a cada término. De este modo, una misma palabra puede tener representaciones diferentes según su significado en cada frase, lo que permite capturar matices semánticos más complejos.

Finalmente, además de las representaciones a nivel de palabra, existen embeddings de frases y documentos, que permiten representar unidades de texto de mayor tamaño mediante un único vector. Estas representaciones son especialmente relevantes en tareas de recuperación de información y búsqueda semántica, ya que facilitan la comparación directa entre consultas y documentos completos en el espacio vectorial.

En conjunto, esta evolución desde embeddings estáticos a contextuales, así como la extensión a unidades de mayor tamaño, ha permitido mejorar

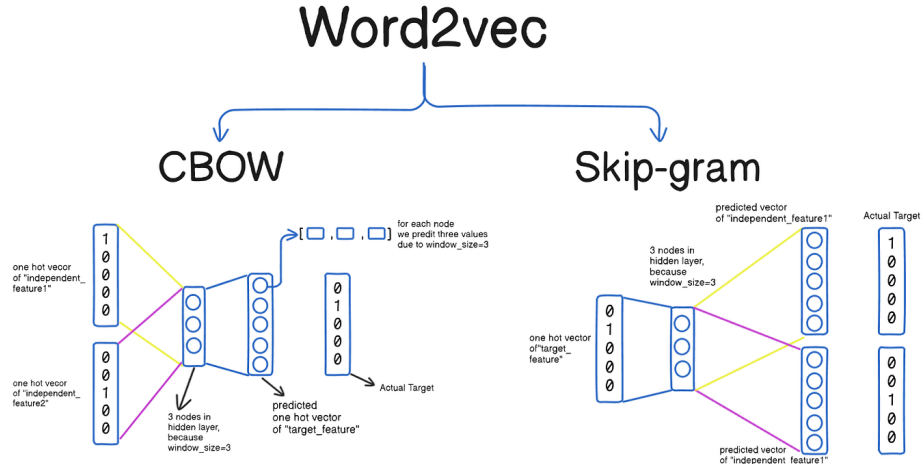


Figura 3.1: Funcionamiento de Word2Vec con sus dos arquitecturas principales [35]

significativamente la capacidad de los sistemas para modelar el lenguaje natural y ha sentado las bases de muchas aplicaciones modernas, especialmente en el ámbito de la recuperación de información.

3.2.2. Similitud y uso en recuperación

Una vez obtenidas las representaciones vectoriales mediante embeddings, resulta necesario definir una medida que permita cuantificar la similitud entre distintos elementos en el espacio vectorial. La métrica más común en este contexto es la **similitud del coseno** [20, 1], que mide el ángulo entre dos vectores independientemente de su magnitud. De este modo, dos vectores con orientación similar presentan un valor cercano a 1, mientras que vectores ortogonales o no relacionados tienden a valores próximos a 0. Esta propiedad la convierte en una herramienta especialmente adecuada para comparar representaciones semánticas.

$$\text{sim}(\mathbf{A}, \mathbf{B}) = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

Donde:

- $\mathbf{A} \cdot \mathbf{B} \rightarrow$ producto escalar de los vectores
- $\|\mathbf{A}\| \|\mathbf{B}\| \rightarrow$ norma (longitud) de cada vector
- $\theta \rightarrow$ ángulo entre los dos vectores

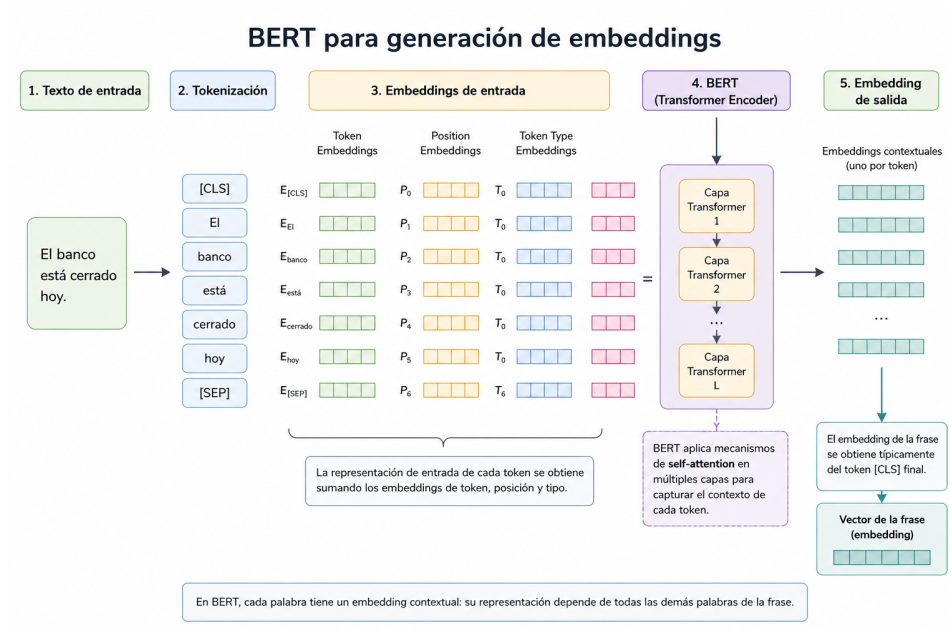


Figura 3.2: BERT para generación de embeddings, imagen generada con IA

Además de la similitud del coseno, existen otras métricas utilizadas en este contexto. Por ejemplo, el **producto escalar** (dot product) se emplea frecuentemente en sistemas de recuperación densa:

$$\text{sim}(\mathbf{A}, \mathbf{B}) = \mathbf{A} \cdot \mathbf{B} = \sum_{i=1}^n A_i B_i$$

Cuando los embeddings están previamente normalizados (es decir, $\|\mathbf{A}\| = \|\mathbf{B}\| = 1$), esta medida resulta equivalente a la similitud del coseno, ya que:

$$\mathbf{A} \cdot \mathbf{B} = \cos(\theta)$$

Asimismo, en enfoques más avanzados, pueden utilizarse modelos de lenguaje para estimar directamente la relevancia entre consulta y documento, como ocurre en los denominados *cross-encoders*. En este caso, la similitud se obtiene a partir de una función aprendida por el modelo:

$$\text{sim}(q, d) = f_{\theta}(q, d)$$

donde f_{θ} representa un modelo de lenguaje parametrizado (por ejemplo, un Transformer), que recibe conjuntamente la consulta q y el documento d y produce una puntuación de relevancia.

No obstante, la similitud del coseno presenta ciertas limitaciones. En particular, cuando los textos a comparar son muy largos, la representación

mediante un único embedding puede perder información relevante debido a la compresión del contenido en un solo vector, lo que puede reducir la precisión de la comparación semántica. Además, al considerar únicamente la orientación de los vectores, el coseno puede ignorar información relacionada con la magnitud, lo que en algunos casos limita su capacidad discriminativa.

El uso de embeddings permite representar consultas y documentos en un espacio vectorial continuo, donde la relevancia se estima mediante la proximidad entre sus representaciones. Este enfoque, conocido como dense retrieval, facilita la recuperación de documentos con significado similar aunque no compartan términos exactos. En la práctica, suele apoyarse en bases de datos vectoriales para realizar búsquedas eficientes en espacios de alta dimensionalidad, y a menudo se combina con métodos tradicionales como BM25 [47] o TF-IDF [19] para mejorar el rendimiento global mediante enfoques híbridos.

3.2.3. Ventajas, limitaciones y usos en RAG

El uso de embeddings en el procesamiento del lenguaje natural presenta numerosas ventajas frente a enfoques tradicionales basados en representaciones discretas. En primer lugar, permiten capturar relaciones semánticas complejas entre palabras, frases o documentos, lo que facilita la recuperación de información más relevante incluso cuando no existe coincidencia exacta de términos. Además, su representación en espacios vectoriales continuos permite aplicar medidas de similitud eficientes, como la similitud del coseno, y utilizar estructuras optimizadas para la búsqueda aproximada de vecinos más cercanos, lo que mejora la escalabilidad en grandes volúmenes de datos.

Otra ventaja importante es su versatilidad, ya que pueden aplicarse tanto a nivel de palabra como de frase o documento, y adaptarse a distintos dominios mediante el uso de modelos preentrenados o ajustados (fine-tuning) [10, 51]. Esto los convierte en una herramienta fundamental en una amplia variedad de tareas de procesamiento del lenguaje natural.

Sin embargo, los embeddings también presentan ciertas limitaciones. Los modelos estáticos pueden perder información contextual relevante, mientras que los embeddings contextuales, aunque más precisos, requieren mayor capacidad computacional y pueden ser más costosos en términos de tiempo de inferencia. Además, la interpretación de los vectores resultantes no es directa, lo que dificulta la explicabilidad de los modelos basados en estas representaciones.

En el contexto de los sistemas de Retrieval-Augmented Generation (RAG) [20, 24], los embeddings desempeñan un papel clave al permitir la recuperación semántica de documentos relevantes para una consulta dada. En estos sistemas, tanto las consultas del usuario como los documentos del corpus se representan como vectores en un espacio común, lo que facilita la identificación de la información más pertinente mediante medidas de similitud.

Posteriormente, esta información recuperada se utiliza como contexto adicional para modelos generativos, mejorando así la precisión y coherencia de las respuestas generadas. Este enfoque híbrido combina las capacidades de recuperación de los embeddings con la potencia generativa de los modelos de lenguaje, dando lugar a sistemas más robustos y efectivos.

3.3. Modelos largos de lenguaje

Los Large Language Models (LLMs) [20] constituyen una de las principales evoluciones recientes en el ámbito del procesamiento del lenguaje natural. Se trata de modelos de aprendizaje profundo diseñados para comprender y generar texto en lenguaje natural a partir de grandes volúmenes de datos. Su funcionamiento se basa en el modelado estadístico del lenguaje, cuyo objetivo es estimar la probabilidad de aparición de una secuencia de palabras dada una secuencia previa.

En términos generales, un LLM aprende a predecir el siguiente token (palabra o subpalabra) en una secuencia, lo que le permite generar texto de manera coherente y contextualmente relevante. Este enfoque, conocido como modelado del lenguaje autoregresivo, constituye la base de muchas de sus capacidades, incluyendo la generación de texto, la respuesta a preguntas o la traducción automática.

Una de las características fundamentales de estos modelos es su entrenamiento sobre grandes corpus de texto, que pueden incluir libros, artículos, páginas web y otros recursos lingüísticos. Este proceso de preentrenamiento permite a los modelos adquirir conocimiento general sobre el lenguaje, incluyendo aspectos sintácticos, semánticos e incluso ciertos patrones de razonamiento. Posteriormente, estos modelos pueden ser adaptados a tareas específicas mediante técnicas de ajuste fino (fine-tuning) [10, 51] o mediante el uso de instrucciones en lenguaje natural.

En conjunto, los LLMs representan un cambio de paradigma en el tratamiento del lenguaje natural, al pasar de enfoques basados en tareas específicas a modelos generales capaces de abordar múltiples problemas mediante un único sistema, lo que ha impulsado el desarrollo de aplicaciones más flexibles y potentes en este ámbito.

3.3.1. Arquitectura base: Transformers

La mayoría de los Large Language Models modernos se basan en la arquitectura Transformers [51]. Este enfoque supuso un cambio significativo respecto a modelos previos como RNNs [20] o LSTMs [20], al prescindir completamente de recurrencia y convoluciones, y basarse exclusivamente en mecanismos de atención para modelar dependencias entre elementos de una secuencia.

El componente fundamental de esta arquitectura es el mecanismo de **self-attention** (Figura 3.3) [20, 51], que permite a cada token de la secuencia atender a todos los demás tokens, capturando así relaciones a larga distancia de manera eficiente. Dado un conjunto de representaciones de entrada, estas se proyectan en tres espacios distintos: consultas (queries), claves (keys) y valores (values), definidos como:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

donde:

- X es la matriz de embeddings de entrada.
- W_Q, W_K, W_V son matrices de parámetros aprendibles.

La atención se calcula mediante el producto escalar entre consultas y claves, seguido de una normalización mediante la función softmax:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

donde:

- d_k es la dimensión de las claves, utilizada como factor de escala para estabilizar los gradientes.

Además, los Transformers emplean **multi-head attention** (Figura 3.3) [20, 51], que consiste en aplicar múltiples mecanismos de atención en paralelo, permitiendo al modelo capturar diferentes tipos de relaciones semánticas. Formalmente, se define como:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

donde cada cabeza de atención se calcula como:

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$

Tras el bloque de atención, cada capa del Transformer incluye una red neuronal completamente conectada aplicada de forma independiente a cada posición (feed-forward network) (Figura 3.3), generalmente definida como:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Estas subcapas se combinan con conexiones residuales y normalización (*layer normalization*), lo que facilita el entrenamiento de modelos profundos y mejora la estabilidad numérica.

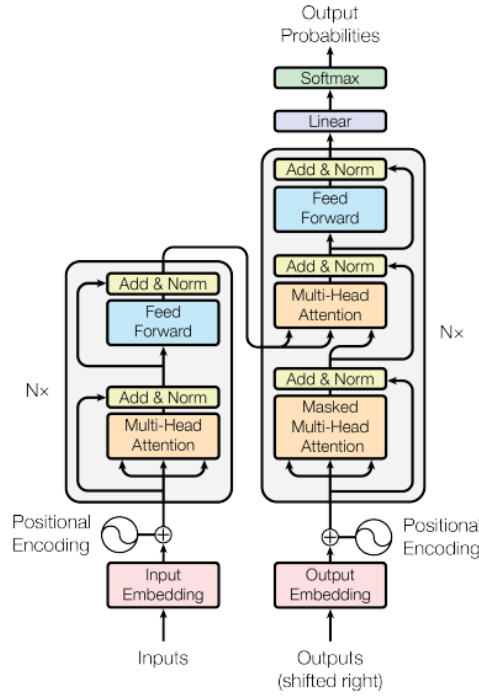


Figura 3.3: Arquitectura básica de un Transformer [51]

Dado que el modelo no incorpora información secuencial de forma explícita, se añaden **codificaciones posicionales** a los embeddings de entrada para introducir información sobre el orden de los tokens. Estas pueden definirse mediante funciones sinusoidales:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

donde:

- pos es la posición del token.
- i la dimensión.

En el caso de los LLMs, es habitual emplear únicamente la parte decodificadora del Transformer, utilizando un mecanismo de atención enmascarada (*masked self-attention*) que impide que cada posición acceda a tokens futuros durante el entrenamiento. Esto garantiza que el modelo aprenda a generar texto de manera autoregresiva.

En conjunto, la arquitectura Transformer proporciona un marco altamente paralelizable y escalable, capaz de modelar dependencias complejas en secuencias largas, lo que la ha convertido en la base de los modelos de lenguaje actuales.

3.3.2. Proceso de entrenamiento y capacidades

El entrenamiento de los Large Language Models se basa en el objetivo de modelar la distribución de probabilidad de secuencias de texto. Dado un conjunto de tokens $x_1, x_2, \dots, x_T$, el modelo aprende a estimar la probabilidad conjunta de la secuencia mediante una descomposición autoregresiva [20, 28]:

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1})$$

Este enfoque implica que, en cada paso, el modelo predice el siguiente token condicionado al contexto previo. Para ello, se utiliza una función de pérdida basada en la entropía cruzada entre la distribución predicha por el modelo y la distribución real de los datos [12]:

$$\mathcal{L} = - \sum_{t=1}^T \log P(x_t \mid x_1, \dots, x_{t-1})$$

Durante el entrenamiento, los tokens de entrada se transforman en embeddings y se procesan a través de múltiples capas Transformer con atención enmascarada, lo que garantiza que la predicción en la posición t dependa únicamente de los tokens anteriores. La salida del modelo en cada posición se proyecta sobre el vocabulario mediante una capa lineal seguida de una función softmax [12]:

$$P(x_t \mid \cdot) = \text{softmax}(h_t W_o + b_o)$$

donde:

- h_t es la representación oculta del modelo en la posición t .
- W_o, b_o son los parámetros de la capa de salida.

El proceso de entrenamiento suele dividirse en dos fases principales. En primer lugar, el **preentrenamiento**, en el que el modelo se entrena sobre grandes corpus de texto no etiquetado con el objetivo de aprender representaciones generales del lenguaje. En esta fase, el modelo adquiere conocimiento sintáctico, semántico y ciertos patrones estadísticos presentes en los datos.

Posteriormente, se puede aplicar una fase de **ajuste fino** (fine-tuning) [10, 51], en la que el modelo se adapta a tareas específicas mediante conjuntos de datos más pequeños y especializados. En el contexto de los LLMs modernos, también es habitual el uso de técnicas como el *Reinforcement Learning from Human Feedback* (RLHF) [6, 37], donde el modelo se optimiza utilizando señales de preferencia humana para mejorar la calidad, coherencia y utilidad de las respuestas generadas.

Como resultado de este proceso de entrenamiento a gran escala, los LLMs desarrollan una amplia variedad de capacidades emergentes. Entre ellas destacan la generación coherente de texto en lenguaje natural, la capacidad de responder preguntas, realizar tareas de resumen, traducción automática o completar código. Estas capacidades no se entrenan necesariamente de forma explícita para cada tarea, sino que emergen del aprendizaje general del modelo sobre grandes cantidades de datos.

Además, los LLMs son capaces de aprovechar el contexto proporcionado en la entrada para adaptar su comportamiento dinámicamente, lo que permite su uso en escenarios de *in-context learning*, donde el modelo resuelve tareas a partir de ejemplos incluidos en el propio prompt sin necesidad de re-entrenamiento. Esta flexibilidad, combinada con su capacidad de generalización, los convierte en herramientas altamente versátiles en el procesamiento del lenguaje natural.

3.3.3. Limitaciones y uso en RAG

A pesar de sus notables capacidades, los Large Language Models [20] presentan diversas limitaciones que afectan tanto a su fiabilidad como a su aplicabilidad en entornos reales. Una de las más relevantes es la tendencia a generar información incorrecta o no verificable, fenómeno conocido como *alucinación* [20]. Esto se debe a que el modelo está optimizado para predecir la secuencia de tokens más probable, no necesariamente la más veraz, lo que puede dar lugar a respuestas plausibles pero erróneas.

Otra limitación importante es la dependencia del conocimiento adquirido durante el preentrenamiento. Dado que los LLMs no tienen acceso directo a información externa en tiempo de inferencia, su conocimiento está acotado por los datos utilizados durante su entrenamiento, lo que implica dificultades para manejar información actualizada o específica de dominios muy concretos. Asimismo, presentan una capacidad limitada para razonar de forma estrictamente lógica o simbólica, especialmente en tareas que requieren múltiples pasos de inferencia rigurosa.

Desde el punto de vista computacional, estos modelos requieren una gran cantidad de recursos tanto para su entrenamiento como para su despliegue, lo que puede dificultar su uso en entornos con restricciones de memoria o latencia. Además, la naturaleza distribuida de sus representaciones internas complica la interpretabilidad, dificultando la comprensión de por qué el modelo genera una determinada respuesta.

Para abordar parte de estas limitaciones, surge el enfoque de **Retrieval-Augmented Generation (RAG)** [24, 20], que combina modelos generativos con mecanismos de recuperación de información externa. En este paradigma, el modelo no depende únicamente de su conocimiento interno, sino que accede dinámicamente a un corpus de documentos relevantes durante el proceso de generación.

El funcionamiento de un sistema RAG puede describirse en varias etapas. En primer lugar, la consulta del usuario se transforma en un embedding utilizando un modelo adecuado. A continuación, se realiza una búsqueda en un índice vectorial para recuperar los documentos más similares en términos semánticos, empleando medidas como la similitud del coseno [20]. Estos documentos recuperados se incorporan como contexto adicional en la entrada del modelo generativo, que finalmente produce una respuesta condicionada tanto por la consulta como por la información recuperada.

Formalmente, este proceso puede interpretarse como una aproximación a la siguiente distribución:

$$P(y \mid x) = \sum_{d \in \mathcal{D}} P(y \mid x, d) P(d \mid x)$$

donde:

- x es la consulta.
- d representa un documento relevante recuperado de un conjunto $\mathcal{D}$.
- y es la respuesta generada.

En la práctica, esta suma se aproxima utilizando un subconjunto de documentos más relevantes.

Este enfoque permite mejorar significativamente la precisión factual de las respuestas, reducir las alucinaciones y facilitar la actualización del conocimiento sin necesidad de reentrenar el modelo. No obstante, introduce nuevos desafíos, como la dependencia de la calidad del sistema de recuperación, la selección adecuada del contexto y el incremento del coste computacional debido a la integración de múltiples componentes.

En conjunto, los sistemas RAG [24, 20] representan una solución efectiva para complementar las capacidades de los LLMs [20], combinando la potencia generativa de estos modelos con el acceso dinámico a información externa, lo que resulta especialmente útil en aplicaciones de recuperación de información y respuesta en dominios abiertos.

Capítulo 4

Estado del arte

En los últimos años, los sistemas de recuperación de información han experimentado una evolución significativa impulsada por el desarrollo de modelos de lenguaje y técnicas de aprendizaje profundo. Esta evolución ha permitido pasar de enfoques tradicionales basados en coincidencia léxica a métodos capaces de modelar similitud semántica mediante representaciones vectoriales densas y arquitecturas neuronales.

En este contexto, han surgido numerosos trabajos de investigación que abordan el problema de la recuperación de información desde diferentes perspectivas, destacando los modelos neuronales de ranking, los sistemas híbridos de búsqueda y, más recientemente, las arquitecturas basadas en Retrieval-Augmented Generation (RAG). Estos enfoques buscan mejorar la calidad de la recuperación y la generación de respuestas, especialmente en escenarios donde el conocimiento debe ser preciso, actualizado y contextualizado.

El presente capítulo revisa los principales avances en este ámbito, analizando tanto los modelos fundamentales de recuperación neuronal como las distintas propuestas de sistemas RAG existentes en la literatura. Asimismo, se estudian las mejoras introducidas en trabajos recientes y su aplicación en sistemas reales, con el objetivo de situar el trabajo realizado dentro del estado actual de la investigación.

4.1. Introducción a RAG

La generación aumentada por recuperación (Retrieval-Augmented Generation, RAG) ha surgido como una de las líneas de investigación más relevantes en el ámbito de los modelos de lenguaje y la recuperación de información. Este enfoque responde a una de las principales limitaciones de los modelos generativos tradicionales: su dependencia exclusiva del conocimiento adquirido durante el entrenamiento y su incapacidad para incorporar información externa de manera dinámica.

El concepto de RAG fue introducido inicialmente como una forma de

combinar modelos de recuperación de información con modelos generativos, permitiendo que las respuestas se fundamenten en documentos externos relevantes recuperados en tiempo de ejecución [24]. A partir de este trabajo inicial, se han desarrollado múltiples variantes y extensiones que buscan mejorar la calidad de la recuperación, la relevancia del contexto proporcionado y la coherencia de las respuestas generadas.

El interés por este tipo de arquitecturas ha crecido de forma significativa debido a su capacidad para reducir problemas como las alucinaciones de los modelos de lenguaje, así como para permitir la incorporación de conocimiento actualizado sin necesidad de reentrenar los modelos. Como resultado, RAG se ha convertido en un componente clave en numerosos sistemas de inteligencia artificial aplicados a tareas de respuesta, búsqueda semántica y asistentes conversacionales.

En la literatura actual, los sistemas RAG se estudian principalmente desde la perspectiva de la mejora del proceso de recuperación, la integración eficiente del contexto en los modelos generativos y la optimización del equilibrio entre precisión, coste computacional y escalabilidad.

4.2. Sistemas clásicos de recuperación neural

Los sistemas de recuperación basados en aprendizaje profundo han supuesto un cambio sustancial respecto a los enfoques tradicionales de recuperación de información, como BM25 [47], al introducir representaciones densas capaces de modelar similitud semántica entre consultas y documentos. Esta línea de investigación ha dado lugar a diferentes familias de modelos que han ido mejorando progresivamente el equilibrio entre eficiencia computacional y precisión en la recuperación.

Uno de los trabajos más influyentes en este ámbito es Dense Passage Retrieval (DPR) [21]. Este modelo introduce una arquitectura basada en bi-encoders que codifica de forma independiente la consulta y el documento mediante redes Transformer. Ambas representaciones se proyectan en un espacio vectorial compartido donde la relevancia se calcula mediante similitud, habitualmente producto escalar. La principal contribución de DPR es demostrar que, mediante un entrenamiento supervisado adecuado con pares (consulta, documento relevante), es posible mejorar significativamente la recuperación semántica en tareas de question answering a gran escala. Además, su diseño permite el uso de técnicas de búsqueda eficiente mediante índices aproximados de vecinos más cercanos (ANN) [15], lo que lo hace altamente escalable.

A pesar de su eficiencia, los bi-encoders [45] como DPR presentan una limitación importante: la interacción entre consulta y documento se produce únicamente a nivel de representación global, sin modelar relaciones finas entre tokens. Para abordar este problema, surgen modelos como ColBERT

[22], que introducen el concepto de *late interaction*. En este enfoque, tanto la consulta como el documento se representan como conjuntos de embeddings a nivel de token, lo que permite calcular la relevancia mediante interacciones más granulares entre ambas secuencias. ColBERT combina esta mayor expresividad con estrategias de indexación que permiten mantener una eficiencia razonable en entornos de gran escala, mejorando la precisión respecto a los bi-encoders clásicos.

Dentro de este conjunto de modelos también se encuentran enfoques como Sentence-BERT (SBERT) [45], que adapta arquitecturas tipo BERT mediante aprendizaje siamés para generar embeddings semánticamente significativos a nivel de frase. SBERT fue uno de los trabajos clave en la consolidación de los bi-encoders como solución práctica para tareas de búsqueda semántica, ya que permite precomputar representaciones de documentos y realizar consultas de forma eficiente.

En paralelo, los sistemas de recuperación neuronal suelen combinarse con modelos de reordenación (*reranking*) basados en cross-encoders [10]. En estos enfoques, la consulta y el documento se procesan conjuntamente dentro de un modelo Transformer, lo que permite capturar interacciones profundas entre ambos textos. Aunque este tipo de modelos no escala bien a colecciones grandes debido a su coste computacional, su elevada precisión los hace especialmente adecuados para la segunda etapa de sistemas de recuperación en dos fases.

En conjunto, estos enfoques representan la transición desde métodos basados en coincidencia léxica hacia sistemas de recuperación semántica basados en aprendizaje profundo. Esta evolución ha sentado las bases de los sistemas modernos de búsqueda y recuperación, y constituye un componente esencial en arquitecturas más avanzadas como los sistemas de Retrieval-Augmented Generation (RAG) [24], donde la calidad del módulo de recuperación condiciona directamente el rendimiento del sistema generativo.

4.3. Evolución de RAG

Desde la introducción del enfoque de Retrieval-Augmented Generation (RAG) por Lewis et al. [24], se han propuesto múltiples extensiones y variantes con el objetivo de mejorar la integración entre los módulos de recuperación y generación. El trabajo original plantea una arquitectura en la que un modelo generativo se condiciona mediante documentos recuperados dinámicamente desde una base de conocimiento externa, permitiendo reducir la dependencia exclusiva del conocimiento paramétrico del modelo.

A partir de esta propuesta inicial, la investigación en este ámbito ha evolucionado hacia sistemas más integrados y sofisticados, donde el proceso de recuperación se incorpora de manera más profunda en el ciclo de generación o incluso en el propio entrenamiento del modelo.

Uno de los primeros enfoques relevantes en esta evolución es REALM (Retrieval-Augmented Language Model) [14], propuesto por Google. A diferencia del RAG original, REALM integra el proceso de recuperación durante el preentrenamiento del modelo, lo que permite que el sistema aprenda de manera conjunta a recuperar documentos relevantes y a utilizarlos para la generación de texto. Esta integración end-to-end supone un avance importante respecto a arquitecturas donde la recuperación se mantiene como un módulo externo fijo.

Posteriormente, el modelo RETRO (Retrieval-Enhanced Transformer) [5], desarrollado por DeepMind, introduce un enfoque escalable de recuperación a gran escala. RETRO incorpora fragmentos de texto recuperados desde una base de datos masiva directamente en el proceso de atención del Transformer, permitiendo al modelo acceder a información externa durante la generación. Este enfoque demuestra que es posible mejorar significativamente el rendimiento de los modelos de lenguaje sin aumentar proporcionalmente el número de parámetros del modelo, apoyándose en la recuperación como fuente externa de conocimiento.

Otra línea de investigación relevante es Atlas [16], que combina entrenamiento supervisado y recuperación dinámica, permitiendo al modelo aprender a utilizar información externa de manera más flexible en tareas de question answering. Atlas demuestra que los modelos pueden ser entrenados específicamente para mejorar su capacidad de recuperación y generación conjunta, alcanzando buenos resultados incluso con menos datos de entrenamiento que otros enfoques tradicionales.

Más recientemente, han surgido propuestas que buscan dotar a los sistemas RAG de mayor autonomía y capacidad de razonamiento sobre el propio proceso de recuperación. Un ejemplo de ello es Self-RAG [3], en el que el modelo aprende a decidir cuándo recuperar información externa, cuándo utilizar su conocimiento interno y cómo evaluar la calidad de las respuestas generadas. Este enfoque introduce un componente de reflexión que permite mejorar la fiabilidad del sistema y reducir la generación de respuestas no fundamentadas.

De forma paralela, técnicas como HyDE (Hypothetical Document Embeddings) [11] han explorado estrategias alternativas para mejorar la recuperación, generando primero documentos hipotéticos a partir de la consulta y utilizando estos como base para la búsqueda en el espacio vectorial. Este enfoque busca mejorar la calidad del retrieval cuando la consulta original es ambigua o poco informativa.

En conjunto, la evolución de los sistemas RAG muestra una tendencia clara hacia la integración progresiva entre recuperación y generación, pasando de arquitecturas modulares simples a sistemas cada vez más unificados, adaptativos y dependientes del contexto. Esta evolución ha permitido mejorar tanto la precisión como la robustez de los modelos, consolidando el enfoque RAG como una de las líneas más activas en la investigación actual

en modelos de lenguaje.

4.4. Mejoras recientes y aplicaciones reales

En los últimos años, los sistemas basados en Retrieval-Augmented Generation han experimentado una evolución significativa orientada a mejorar tanto la calidad de la recuperación como la precisión y fiabilidad de las respuestas generadas. Estas mejoras han dado lugar a arquitecturas más robustas y eficientes, capaces de adaptarse a entornos reales donde la escalabilidad, la actualización del conocimiento y la reducción de errores resultan aspectos fundamentales.

Una de las principales líneas de mejora consiste en el uso de sistemas híbridos de recuperación, que combinan métodos tradicionales basados en coincidencia léxica, como BM25 [47], con modelos densos basados en embeddings. Este enfoque híbrido [2, 25] permite aprovechar simultáneamente la precisión terminológica de los sistemas clásicos y la capacidad semántica de los modelos neuronales. Como resultado, los sistemas híbridos suelen ofrecer mejores resultados que los enfoques puramente léxicos o puramente densos, especialmente en consultas complejas o ambiguas.

Además, muchos sistemas modernos incorporan etapas de reranking neuronal [32] mediante modelos cross-encoder, capaces de analizar conjuntamente la consulta y los documentos recuperados. Este tipo de modelos permite refinar el conjunto de resultados iniciales y mejorar significativamente la relevancia del contexto proporcionado al modelo generativo. Aunque estas técnicas incrementan el coste computacional, su impacto positivo sobre la calidad final de las respuestas ha favorecido su adopción en sistemas de producción.

Otra línea reciente de investigación se centra en el uso de estrategias de reformulación de consultas (*query rewriting*) y recuperación multi-hop. En estos enfoques, el sistema puede generar consultas intermedias o realizar múltiples etapas de recuperación para obtener información relacionada distribuida en diferentes documentos. Este tipo de arquitecturas resulta especialmente útil en tareas que requieren razonamiento complejo o integración de información procedente de múltiples fuentes.

Asimismo, han comenzado a surgir sistemas conocidos como *Agentic RAG* [24], donde modelos de lenguaje con capacidades de planificación y razonamiento controlan dinámicamente el proceso de recuperación. En estos sistemas, el modelo puede decidir cuándo recuperar información adicional, qué fuentes consultar o cómo validar las respuestas generadas. Este enfoque representa una evolución hacia sistemas más autónomos y adaptativos, aunque todavía presenta importantes desafíos relacionados con el coste computacional y la fiabilidad.

En paralelo a estas mejoras arquitectónicas, también se ha producido un

importante desarrollo de herramientas y plataformas destinadas a facilitar la implementación de sistemas RAG en entornos reales. Frameworks como LangChain o LlamaIndex permiten construir pipelines de recuperación y generación de forma modular, mientras que bases de datos vectoriales como FAISS, Pinecone, Weaviate o Milvus facilitan la indexación y búsqueda eficiente sobre grandes colecciones de embeddings [45].

Las arquitecturas RAG han comenzado además a aplicarse en numerosos dominios reales. En el ámbito jurídico, estos sistemas se utilizan para la búsqueda y análisis de jurisprudencia y documentación legal, permitiendo generar respuestas fundamentadas en normativa específica. En medicina y biomedicina, RAG se emplea para acceder a literatura científica y asistir en tareas de consulta clínica o recuperación de información biomédica especializada.

Por otro lado, en entornos empresariales, los sistemas RAG se utilizan ampliamente en asistentes conversacionales, motores de búsqueda corporativos y sistemas de soporte técnico, donde permiten acceder dinámicamente a documentación interna y bases de conocimiento actualizadas. Este tipo de aplicaciones resulta especialmente relevante en organizaciones donde la información cambia con frecuencia y no puede integrarse fácilmente mediante reentrenamiento continuo de modelos.

En conjunto, las mejoras recientes en recuperación híbrida, reranking, razonamiento multi-hop y sistemas agentivos muestran una evolución progresiva de las arquitecturas RAG hacia soluciones más precisas, adaptativas y aplicables en escenarios reales. Estas tendencias reflejan además el creciente interés de la comunidad investigadora e industrial por sistemas capaces de combinar generación de lenguaje natural con acceso dinámico y fiable a conocimiento externo [24].

4.5. Comparación con el trabajo realizado

Tras analizar los principales enfoques presentes en la literatura, puede observarse que la mayoría de sistemas RAG modernos combinan técnicas de recuperación semántica, modelos híbridos y modelos generativos de gran tamaño con el objetivo de mejorar la precisión y fundamentación de las respuestas.

El trabajo desarrollado en este proyecto sigue esta línea general mediante la implementación y comparación de diferentes arquitecturas RAG aplicadas sobre documentación parlamentaria. En concreto, se han desarrollado varios sistemas diferenciados según la estrategia de recuperación utilizada.

En primer lugar, se implementó un sistema basado exclusivamente en recuperación léxica mediante BM25 [47], utilizado como línea base tradicional. Este enfoque permite recuperar documentos relevantes a partir de coincidencia de términos y constituye uno de los métodos más utilizados

históricamente en recuperación de información.

Posteriormente, se desarrolló un sistema de recuperación semántica basado en embeddings densos utilizando el modelo *intfloat/multilingual-e5-base* [21]. Este enfoque permite representar consultas y documentos en un espacio vectorial compartido, facilitando la recuperación de información relacionada semánticamente incluso cuando no existe coincidencia exacta de términos.

Además, se implementaron dos variantes híbridas. La primera utiliza el sistema híbrido [25, 2] proporcionado de forma nativa por OpenSearch, mientras que la segunda emplea una estrategia híbrida manual basada en la combinación de resultados BM25 y recuperación vectorial mediante Reciprocal Rank Fusion (RRF) [7]. Este último enfoque permite combinar explícitamente las ventajas de la recuperación léxica y semántica, integrando ambas puntuaciones dentro de un ranking unificado.

En todos los sistemas implementados, la generación de respuestas se realiza mediante el modelo generativo *Mistral-7B-Instruct-v0.1* [17], utilizado como componente común para garantizar que las diferencias observadas dependan principalmente de la estrategia de recuperación empleada y no del modelo generativo.

A diferencia de arquitecturas avanzadas presentes en la literatura, como REALM o RETRO, el sistema desarrollado mantiene una arquitectura modular desacoplada entre recuperación y generación. Esta decisión permite facilitar la experimentación y evaluar de forma independiente el impacto de cada técnica de retrieval sobre la calidad final de las respuestas.

Asimismo, aunque el sistema no incorpora mecanismos avanzados de razonamiento multi-hop o agentes autónomos, sí integra varias de las tendencias actuales observadas en sistemas RAG modernos, como recuperación híbrida, búsqueda semántica mediante embeddings y fusión de rankings.

Otro aspecto relevante del trabajo realizado es el uso de documentos parlamentarios reales como base de conocimiento. Esto introduce características particulares relacionadas con el lenguaje formal, la presencia de metadatos estructurados y la necesidad de recuperar información precisa dentro de grandes volúmenes documentales.

Finalmente, la comparación entre las distintas arquitecturas implementadas permite analizar de forma experimental el impacto de las estrategias de recuperación léxica, semántica e híbrida sobre la calidad de las respuestas generadas. Esto permite estudiar cómo las técnicas descritas en el estado del arte se comportan en un dominio específico como el parlamentario, caracterizado por documentos extensos, lenguaje formal y abundante información contextual.

Capítulo 5

Materiales y métodos

En este capítulo se describen los materiales, herramientas y metodologías empleadas para el desarrollo y evaluación de los sistemas RAG [24, 20] implementados en este trabajo. En primer lugar, se presenta el conjunto de datos utilizado, compuesto por documentación parlamentaria procesada y estructurada para su posterior indexación y recuperación.

Posteriormente, se describen las métricas utilizadas para evaluar el rendimiento de los distintos sistemas desarrollados, tanto desde el punto de vista de la recuperación de información como de la calidad de las respuestas generadas [48].

Finalmente, se detalla la configuración experimental utilizada durante las pruebas, incluyendo los modelos empleados, las estrategias de recuperación implementadas y los parámetros principales del sistema. Todo ello permite establecer un entorno de evaluación homogéneo que facilite la comparación entre los distintos enfoques estudiados.

5.1. Conjunto de datos

El rendimiento de un sistema Retrieval-Augmented Generation depende en gran medida de la calidad y características de la base de conocimiento utilizada durante el proceso de recuperación. Por este motivo, resulta fundamental disponer de un conjunto de datos adecuadamente estructurado y adaptado al dominio específico sobre el que se desea realizar la búsqueda y generación de respuestas.

En este trabajo se ha utilizado un conjunto de documentos parlamentarios reales, compuesto por intervenciones, iniciativas y sesiones parlamentarias organizadas junto con distintos metadatos asociados. El corpus original está formado por **5258** documentos, que posteriormente han sido procesados y fragmentados en unidades de menor tamaño (*chunks*) con el objetivo de facilitar su indexación y recuperación mediante técnicas tanto léxicas como semánticas.

A continuación, se describen las características principales del conjunto de datos empleado, el proceso de preprocesamiento realizado y la estructura final utilizada para la construcción de los distintos sistemas RAG desarrollados.

5.2. Preprocesamiento e indexación

El proceso de construcción del sistema de recuperación se basa en una fase de preprocesamiento y otra de indexación de los documentos parlamentarios, cuyo objetivo es transformar los archivos XML originales en representaciones estructuradas y recuperables tanto a nivel léxico como semántico.

Este proceso se compone de tres etapas principales:

- (i) La extracción y estructuración de los documentos a partir del formato XML.
- (ii) La fragmentación del texto en unidades de menor tamaño o *chunks*.
- (iii) La indexación de los datos resultantes en sistemas de recuperación basados en OpenSearch, tanto en su variante léxica como vectorial.

5.2.1. Extracción y estructuración de documentos

En esta etapa se realiza la extracción y estructuración de los documentos parlamentarios a partir de su formato original en XML. Cada archivo representa una iniciativa parlamentaria con una estructura jerárquica compuesta por metadatos generales y un conjunto de intervenciones.

El procesamiento se lleva a cabo mediante un analizador XML basado en *ElementTree* [40], que permite recorrer la estructura del documento y acceder a sus distintos niveles de información. En primer lugar, se extraen los metadatos asociados a cada iniciativa, incluyendo la legislatura, el órgano parlamentario, la fecha, el tipo de sesión y el tipo de iniciativa.

Posteriormente, se procesa el texto, organizado en intervenciones realizadas por distintos participantes. Cada intervención contiene a su vez una serie de párrafos que conforman el discurso completo. Estos párrafos se concatenan para reconstruir el texto continuo de cada intervención.

Como resultado de esta transformación, los 5258 archivos XML originales se descomponen en un total de **58426** documentos independientes, donde cada documento corresponde a una intervención individual acompañada de los metadatos de contexto asociados a la iniciativa parlamentaria de la que procede. Esto permite representar cada intervención como una unidad documental autónoma, facilitando tanto su indexación como su recuperación posterior.

Estos documentos estructurados se indexan posteriormente en OpenSearch en el índice *iniciativas_flat*, donde cada intervención queda almacenada junto con sus metadatos asociados, permitiendo su consulta eficiente en las fases posteriores del sistema, es importante aclarar que aunque el índice se llame *iniciativas_flat*, los **58426** documentos corresponden cada uno a una intervención individual.

La Tabla 5.1 muestra un ejemplo de uno de los documentos generados tras el proceso de extracción y estructuración.

Como resultado final, cada intervención se transforma en una unidad textual estructurada, asociada a sus metadatos correspondientes, lo que permite su posterior fragmentación en unidades de tamaño controlado e indexación en los distintos sistemas de recuperación implementados.

Campo	Valor
Archivo	DSCA080028.1.xml
Legislatura	VIII
Número de diario	28
Órgano	Comisión de Salud
Tipo de sesión	Ordinaria
Fecha	21 de mayo de 2008
Tipo de iniciativa	Comparecencias
Presidencia	Ilma. Sra. Dña. Raquel Arenal Catena
Interviniente	La señora CORREDERA QUINTANA
Extracto	Comparecencias de la Excm. Sra. Consejera de Salud para informar acerca de las líneas de actuación de la Consejería en la presente legislatura.
Discurso	«Muchas gracias. Bueno, en primer lugar, agradecer la información de la Consejera y que haya respondido a algunos de los planteamientos. Alguna cosa se ha quedado, pero poca. Y antes de hablar con la Consejera...»

Cuadro 5.1: Ejemplo de documento generado tras el proceso de extracción y estructuración de una intervención parlamentaria.

5.2.2. Fragmentación en chunks

Los fragmentos generados en la etapa de preprocesamiento son almacenados en un índice de OpenSearch denominado *iniciativas_chunks2*. Cada intervención textual es dividida en fragmentos de 300 tokens, denominado *chunks_size*, con el objetivo de adaptar el contenido a las limitaciones del modelo generativo y mejorar la granularidad de la recuperación. Como resultado de este proceso de segmentación, se obtienen un total de **162638** chunks.

La fragmentación se realiza utilizando el tokenizador del modelo *Mistral-7B-Instruct-v0.1*, garantizando coherencia entre el proceso de chunking y el posterior uso del texto en el sistema RAG.

Cada fragmento se trata como un documento independiente asociado a los metadatos parlamentarios correspondientes.

Es importante destacar que este proceso de preprocesamiento se aplica de forma conjunta a los 5258 documentos XML originales, generando dos representaciones distintas del corpus a partir de un mismo flujo de extracción: por un lado, las intervenciones estructuradas se almacenan en el índice *iniciativas_flat*; por otro, estas mismas intervenciones son posteriormente fragmentadas en tamaño *chunks_size* e indexadas en *iniciativas_chunks2*. Fragmentando solamente el discurso y posteriormente añadiendo los metadatos del archivo original para complementar la información.

De este modo, ambos índices comparten el mismo origen y preprocesamiento inicial, diferenciándose únicamente en el nivel de granularidad del texto almacenado, lo que permite evaluar estrategias de recuperación a distintos niveles de segmentación.

Esta estructura permite la aplicación del modelo de recuperación basado en BM25, facilitando la búsqueda léxica sobre unidades textuales de tamaño reducido en lugar de documentos completos, lo que mejora la precisión del sistema de recuperación.

El proceso de fragmentación e indexación se formaliza en el Algoritmo 1.

5.2.3. Indexación semántica mediante embeddings

A partir de los fragmentos almacenados en el índice léxico, se genera una representación vectorial utilizando el modelo de embeddings multilingüe *intfloat/multilingual-e5-base* [52], implementado mediante la librería *SentenceTransformers* [45].

Para cada fragmento textual se calcula un embedding denso de 768 dimensiones, normalizado para facilitar la comparación mediante similitud coseno. Siguiendo las recomendaciones del modelo E5, se utiliza el prefijo **passage:** para los documentos indexados y **query:** para las consultas durante la fase de recuperación.

Los embeddings generados se almacenan en un segundo índice de OpenSearch denominado *iniciativas_chunks2_emb*, configurado como índice vectorial mediante el tipo **knn_vector**. Para la búsqueda eficiente de vecinos más cercanos se emplea el algoritmo HNSW (Hierarchical Navigable Small World) [26] como método de aproximación de vecinos más cercanos, junto con el motor FAISS [18] como backend de ejecución, permitiendo así una recuperación eficiente en grandes colecciones documentales bajo el paradigma de Approximate Nearest Neighbors (ANN) [15].

La Tabla 5.2 muestra un ejemplo de uno de los documentos generados tras el proceso de vectorización.

Esta estructura permite la implementación de recuperación semántica eficiente sobre grandes colecciones documentales y constituye la base de los

sistemas RAG basados en embeddings [21] e híbridos [25] desarrollados en este trabajo.

Campo	Valor
Índice	iniciativas_chunks2_emb
Archivo	DSCA080032_1.xml
Legislatura	VIII
Número de diario	32
Órgano	Comisión de Obras Públicas y Transportes
Tipo de sesión	Ordinaria
Fecha	4 de junio de 2008
Tipo de iniciativa	Comparecencias
Interviniente	El señor GARCÍA RODRÍGUEZ
Texto (chunk)	«iría que demos un saltito más y que, cada vez que se haga un proyecto o se haga una propuesta legislativa... sobre la influencia en las desigualdades en general...»
Embedding (768 dim.)	[0.0241, 0.0562, -0.0289, 0.0008, 0.0203, -0.0697, ... , -0.0251, 0.0660]

Cuadro 5.2: Ejemplo de documento vectorizado en el índice *iniciativas_chunks2_emb*.

La generación de embeddings e indexación vectorial se formaliza en el Algoritmo 2.

No se ha realizado un proceso de *fine-tuning* [10, 51] de los modelos de *embeddings* ni del modelo de lenguaje empleado. Aunque el corpus parlamentario disponible constituye una fuente de información especializada que podría utilizarse para continuar el entrenamiento de estos modelos mediante técnicas de aprendizaje auto-supervisado, la realización de dicho proceso requiere una capacidad computacional considerable, especialmente en términos de memoria GPU, tiempo de entrenamiento y ajuste de hiperparámetros.

Dado que el objetivo principal de este trabajo es evaluar el comportamiento de distintas estrategias de recuperación dentro de una arquitectura RAG, se ha optado por emplear modelos preentrenados de propósito general. Esta decisión permite aislar el efecto de los distintos mecanismos de *retrieval* y realizar una comparación objetiva entre ellos, evitando que las mejoras derivadas de un entrenamiento adicional condicionen los resultados experimentales.

No obstante, el ajuste específico de los modelos sobre el corpus parlamentario constituye una línea de trabajo futura de gran interés. Un proceso de *fine-tuning* permitiría adaptar tanto las representaciones vectoriales como la capacidad generativa del modelo al vocabulario, la terminología jurídica y el estilo lingüístico característicos de las intervenciones parlamentarias, con el potencial de mejorar tanto la recuperación de documentos relevantes como la calidad de las respuestas generadas.

5.2.4. Pseudocódigo del proceso de indexación

El conjunto de datos se organiza en tres niveles de indexación en OpenSearch:

- *iniciativas_flat*: contiene las intervenciones parlamentarias estructuradas con sus metadatos.
- *iniciativas_chunks2*: almacena las intervenciones fragmentadas en *chunks_size* con sus metadatos.
- *iniciativas_chunks2_emb*: incluye los mismos fragmentos enriquecidos con su representación vectorial además de los metadatos.

A continuación se presenta una descripción algorítmica del proceso completo de preprocesamiento e indexación de los documentos parlamentarios.

Algorithm 1 Fragmentación e indexación léxica

```

1: for cada intervención do
2:   Dividir texto en chunks_size
3:   for cada chunk do
4:     Crear documento con metadatos
5:     Insertar en índice OpenSearch (BM25)
6:   end for
7: end for

```

Algorithm 2 Generación de embeddings e indexación vectorial

```

1: for cada chunk en el índice léxico do
2:   Añadir prefijo passage:
3:   Generar embedding con modelo E5
4:   Normalizar embedding
5:   Almacenar en índice vectorial OpenSearch
6: end for

```

5.3. Métricas

Para la evaluación de los sistemas RAG desarrollados en este trabajo, se han considerado tanto métricas orientadas a la calidad del proceso de recuperación de información como métricas destinadas a evaluar la calidad de las respuestas generadas por el modelo [48]. De este modo, se obtiene una evaluación completa del comportamiento del sistema en sus dos etapas principales: retrieval y generación.

5.3.1. Métricas de recuperación

Las métricas de recuperación permiten evaluar la capacidad del sistema para localizar correctamente los fragmentos relevantes dentro del conjunto de documentos indexados. En este trabajo se emplean métricas clásicas de Information Retrieval [?] calculadas a nivel de *chunk*.

- **Hit Rate:** indica la proporción de consultas en las que al menos uno de los fragmentos relevantes aparece entre los K primeros resultados recuperados. Esta métrica permite evaluar si el sistema es capaz de recuperar información útil en el conjunto superior de resultados.
- **Precision:** mide la proporción de fragmentos recuperados en los K primeros resultados que son realmente relevantes. Formalmente, se define como:

$$\text{Precision} = \frac{\text{número de chunks relevantes recuperados en } K}{K}$$

- **Recall:** cuantifica la proporción de fragmentos relevantes que han sido recuperados dentro de los K primeros resultados. Se define como:

$$\text{Recall} = \frac{\text{chunks relevantes recuperados en } K}{\text{total de chunks relevantes}}$$

Estas métricas permiten analizar el comportamiento del sistema de recuperación desde la perspectiva tanto de la precisión como de la cobertura de los resultados obtenidos.

5.3.2. Métricas de evaluación de generación

Además de la calidad de la recuperación, es necesario evaluar la calidad de las respuestas generadas [24] por el modelo generativo condicionado al contexto recuperado. Para ello, se utilizan métricas basadas en evaluación automática mediante un modelo evaluador (*LLM-as-a-judge* [53, 13]).

- **Context Relevance:** mide en qué medida el contexto recuperado contiene información útil para responder correctamente a la pregunta formulada.
- **Faithfulness:** evalúa si la respuesta generada está fielmente respaldada por el contexto proporcionado, es decir, si evita la introducción de información no soportada por los documentos recuperados.
- **Answer Relevance:** mide el grado en que la respuesta generada responde de forma directa y adecuada a la pregunta del usuario.

Estas métricas se calculan en una escala de 1 a 5 mediante la utilización de un modelo generativo que actúa como evaluador automático, permitiendo una valoración de la calidad de las respuestas generadas en el sistema RAG.

5.4. Configuración experimental

En esta sección se describe el entorno experimental empleado para la evaluación de los distintos sistemas RAG desarrollados en este trabajo. Se detallan los componentes principales del sistema, incluyendo el modelo generativo, el motor de recuperación, el conjunto de datos de evaluación y los principales parámetros de configuración utilizados.

El objetivo de esta configuración es garantizar la reproducibilidad de los experimentos y establecer un entorno controlado que permita realizar comparaciones consistentes entre los diferentes enfoques de recuperación y generación evaluados.

5.4.1. Entorno de ejecución

Todos los experimentos realizados en este trabajo fueron ejecutados con un equipo portátil con sistema operativo Window 10, en un entorno local equipado con el siguiente hardware:

- Procesador Intel Core i7-10750H a 2.60 GHz, con 6 núcleos y 12 hilos de ejecución.
- Memoria RAM DDR4 de 16 GB a 2667 MHz.
- Tarjeta gráfica NVIDIA GeForce RTX 2060 con 6 GB de memoria VRAM.
- Unidad de almacenamiento SSD NVMe PCIe 3.0 x4 de 1 TB.

El entorno software utilizado se basa en Python 3.11.1 junto con CUDA 12.2 y el controlador NVIDIA versión 537.34 para la aceleración mediante GPU.

La aceleración hardware se empleó tanto para la ejecución del modelo generativo como para la generación de embeddings, utilizando la librería *PyTorch* sobre CUDA. El sistema fue desarrollado principalmente utilizando las librerías *transformers*, *torch*, *opensearch-py* y *numpy*.

5.4.2. Modelo generativo y sistema de recuperación

En esta sección se describen los componentes principales que conforman el sistema Retrieval-Augmented Generation (RAG) desarrollado, centrados en el proceso de recuperación de información y en el modelo generativo encargado de producir las respuestas finales. El sistema combina distintas estrategias de recuperación sobre una colección de documentos parlamentarios estructurados, junto con un modelo de lenguaje de gran tamaño ejecutado localmente.

Sistema de recuperación

El sistema de recuperación se ha implementado utilizando OpenSearch como motor principal de indexación y búsqueda. Los documentos parlamentarios han sido fragmentados previamente en unidades textuales o chunks, los cuales constituyen la unidad básica de recuperación.

Se han evaluado diferentes estrategias de recuperación con el objetivo de analizar su impacto en el rendimiento del sistema RAG.

Recuperación léxica basada en BM25

En primer lugar, se emplea un enfoque de recuperación léxica basado en el algoritmo BM25 [47], implementado directamente en OpenSearch. Este método realiza la búsqueda sobre el campo textual de los fragmentos mediante coincidencia de términos, asignando una puntuación en función de la frecuencia de aparición y la relevancia inversa en el conjunto de documentos.

Este enfoque constituye la línea base del sistema de recuperación y permite capturar coincidencias exactas entre la consulta y el contenido de los documentos.

Recuperación semántica basada en embeddings

En segundo lugar, se implementa un sistema de recuperación semántica basado en embeddings densos generados con el modelo *intfloat/multilingual-e5-base* [52]. Este modelo transforma tanto consultas como fragmentos de texto en representaciones vectoriales de 768 dimensiones, normalizadas para permitir la comparación mediante similitud coseno.

La búsqueda se realiza mediante un índice vectorial en OpenSearch utilizando el algoritmo k-nearest neighbors (kNN) [8], apoyado en estructuras de indexación eficientes como HNSW [26]. Siguiendo la metodología recomendada por el modelo E5, se emplean los prefijos query: para las consultas y passage: para los documentos.

Este enfoque permite capturar similitudes semánticas incluso cuando no existe coincidencia léxica directa entre la pregunta y los documentos.

Recuperación híbrida

Se han evaluado dos estrategias de recuperación híbrida que combinan información léxica y semántica.

Por un lado, OpenSearch proporciona una implementación nativa de consultas híbridas, en las que se combinan simultáneamente los resultados de BM25 y kNN. En este caso, el propio motor se encarga de la fusión de resultados y de la agregación de puntuaciones.

Por otro lado, se implementa un enfoque basado en Reciprocal Rank Fusion (RRF) [7], en el cual las listas de resultados obtenidas mediante BM25 y recuperación vectorial se combinan de forma explícita. En este caso, cada documento recibe una puntuación inversamente proporcional a su posición

en cada ranking, según la siguiente expresión:

$$RRF(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

donde $r(d)$ representa la posición del documento en cada lista de resultados y k es un parámetro de suavizado fijado a 60. Finalmente, se seleccionan los K documentos con mayor puntuación agregada.

Modelo generativo

El modelo generativo encargado de producir las respuestas es *Mistral-7B-Instruct-v0.1* [17], un modelo de lenguaje de propósito general afinado para seguir instrucciones. Este modelo recibe como entrada la pregunta del usuario junto con los fragmentos recuperados por el sistema de recuperación.

La inferencia se realiza de manera local mediante cuantización en 4 bits utilizando la librería *BitsAndBytes* [9], lo que permite reducir significativamente el consumo de memoria sin afectar de forma sustancial al rendimiento del modelo. Esta decisión resulta necesaria debido a las limitaciones de recursos computacionales disponibles, ya que la cuantización reduce de forma considerable los requisitos de memoria GPU, permitiendo la ejecución del modelo en hardware con capacidad limitada.

Durante la generación de respuestas se emplean los siguientes parámetros:

- Temperatura: controla el nivel de aleatoriedad en la generación del texto, de forma que valores más altos favorecen respuestas más diversas y creativas, mientras que valores más bajos producen salidas más deterministas y consistentes.
- Top-p: también conocido como nucleus sampling, define el conjunto de tokens candidatos a partir de la probabilidad acumulada, filtrando las continuaciones menos probables para equilibrar diversidad y coherencia en la generación.
- Número máximo de tokens generados: limita la longitud máxima de la respuesta producida por el modelo, controlando su extensión y el coste computacional asociado.
- Longitud máxima de contexto: establece el número máximo de tokens de entrada que el modelo puede procesar simultáneamente, incluyendo la consulta del usuario y los fragmentos recuperados.
- Modo de muestreo (`do_sample`): activa la generación estocástica basada en muestreo probabilístico en lugar de una decodificación determinista, permitiendo introducir variabilidad en las respuestas generadas

y habilitando el uso conjunto de parámetros como la temperatura y top-p.

Estos parámetros controlan el grado de creatividad y coherencia de las respuestas generadas, equilibrando la diversidad con la fidelidad al contexto proporcionado.

5.4.3. Dataset de evaluación

Con el objetivo de evaluar el rendimiento de los distintos sistemas de Recuperación Aumentada por Generación (RAG), se ha construido un conjunto de evaluación específico a partir de los diarios de sesiones parlamentarias previamente procesados e indexados.

Cabe destacar que el corpus no dispone de etiquetas, metadatos semánticos o anotaciones realizadas por expertos que indiquen la relevancia de los documentos respecto a un tema o consulta dada. Esta ausencia de juicios de relevancia dificulta la aplicación de métodos de evaluación tradicionales basados en ground truth previamente definido, por lo que se ha optado por la generación de un conjunto de evaluación propio de forma semiautomática.

Proceso de generación del Dataset

La construcción del conjunto de evaluación se lleva a cabo de forma semiautomática a partir de los fragmentos documentales almacenados en OpenSearch.

En primer lugar, se recuperan todos los *chunks* indexados y se agrupan según el documento de origen. Posteriormente, los fragmentos de cada documento se ordenan de acuerdo con su identificador (*chunk_id*) para preservar la secuencia original del texto.

A partir de esta organización, se genera un conjunto de candidatos formado por grupos de dos o tres fragmentos consecutivos pertenecientes al mismo documento. Estos grupos se utilizan como contexto para la creación de preguntas que requieren integrar información distribuida en varios fragmentos, simulando así escenarios de recuperación más exigentes que aquellos basados en un único fragmento.

Una vez construido el conjunto de candidatos, se selecciona aleatoriamente una muestra de contextos sobre los que se generaran las preguntas de evaluación. Para cada contexto seleccionado se emplea el modelo *Mistral-7B-Instruct-v0.1* [17], que genera automáticamente una pregunta cuya respuesta puede obtenerse a partir de la información proporcionada. A continuación, utilizando el mismo contexto y la pregunta generada, el modelo produce una respuesta de referencia (*ground truth answer*), que posteriormente se emplea para evaluar la calidad de las respuestas generadas por los distintos sistemas RAG.

Finalmente, se realiza una revisión manual del conjunto generado con el objetivo de eliminar preguntas ambiguas, redundantes o cuya respuesta no estuviera suficientemente respaldada por los fragmentos asociados. Este proceso permite obtener un conjunto de evaluación más consistente y adecuado para comparar el rendimiento de las diferentes estrategias de recuperación estudiadas.

Puede verse el proceso seguido en el Algoritmo 3

Cada instancia del dataset contiene los siguientes campos:

- **query**: pregunta utilizada durante la evaluación.
- **archivo**: identificador del documento original del que se extrajeron los fragmentos.
- **question_type**: tipo de pregunta generada.
- **relevant_chunks**: identificadores de los fragmentos considerados relevantes para responder la consulta.
- **ground_truth_answer**: respuesta de referencia generada a partir del contexto seleccionado.

En cuanto al campo *question_type*, en el conjunto de evaluación generado únicamente se considera la categoría *consecutive*. Este tipo de pregunta se construye a partir de la selección de varios fragmentos (*chunks*) consecutivos pertenecientes al mismo documento, los cuales se utilizan como contexto conjunto para la generación tanto de la pregunta como de la respuesta de referencia. De este modo, se fuerza a que la consulta requiera la integración de información distribuida en múltiples fragmentos contiguos, aumentando la complejidad del escenario de recuperación y simulando situaciones más realistas dentro del dominio parlamentario.

Además dado que cada instancia del conjunto de evaluación está construida a partir de 2-3 *chunks* consecutivos que forman el contexto de referencia, el valor de k en la fase de recuperación tiene un impacto directo en la evaluación. En el caso de $k = 3$, el sistema recupera un número de documentos equivalente al tamaño del contexto relevante mínimo, lo que implica que el *recall* queda condicionado a la capacidad del sistema de recuperar exactamente esos fragmentos específicos dentro de las tres primeras posiciones. Por otro lado, al incrementar k a 5, se amplía el conjunto de candidatos recuperados, lo que aumenta la probabilidad de incluir los tres *chunks* relevantes dentro del resultado, favoreciendo el *recall*. Sin embargo, este aumento también introduce más fragmentos no relevantes, lo que tiende a reducir la *precision*. En consecuencia, la estructura del dataset basada en ventanas de 2-3 *chunks* consecutivos hace que el valor de k influya directamente en el equilibrio entre cobertura y ruido en la evaluación.

El conjunto final de evaluación está compuesto por 40 preguntas distribuidas entre diferentes documentos parlamentarios.

Algorithm 3 Generación automática del dataset de evaluación

- 1: Recuperar todos los fragmentos almacenados en OpenSearch
 - 2: Agrupar los fragmentos por documento original
 - 3: **for** cada documento **do**
 - 4: Ordenar los fragmentos por identificador
 - 5: **for** cada posición del documento **do**
 - 6: Seleccionar aleatoriamente una secuencia de 2 o 3 fragmentos consecutivos
 - 7: Construir el contexto concatenando dichos fragmentos
 - 8: Generar una pregunta usando Mistral-7B-Instruct
 - 9: Generar una respuesta utilizando únicamente el contexto seleccionado
 - 10: Guardar:
 - pregunta generada
 - documento de origen
 - fragmentos relevantes
 - respuesta de referencia
 - 11: **end for**
 - 12: **end for**
 - 13: Revisar manualmente las preguntas generadas
 - 14: Eliminar ejemplos ambiguos o incorrectos
 - 15: Construir el conjunto final de evaluación
-

Ejemplos del Dataset

Campo	Valor
Archivo	DSCA080059_5.xml
Tipo de pregunta	Consecutive
Chunks relevantes	[5, 6, 7]
Pregunta	¿Cómo puede la investigación científica garantizar la representatividad genética de las poblaciones de avutardas en el Alto Guadiato?
Respuesta de referencia (<i>ground truth</i>)	El uso de la investigación científica alrededor de las poblaciones de avutardas en el Alto Guadiato puede garantizar la representatividad genética de la especie. Se ha demostrado que la subpoblación de avutardas del noroeste de Córdoba es la única viable a medio y largo plazo, porque mantiene un intercambio de individuos con población del sur de Extremadura y tiene un contingente importante en la especie.

Campo	Valor
Archivo	DSCA080034_1.xml
Tipo de pregunta	Consecutive
Chunks relevantes	[10, 11, 12]
Pregunta	¿Cómo las empresas en Andalucía pueden garantizar el mantenimiento de sus empleos y la no destrucción de sus empresas, mientras cumplen con las normas de acceso a la vivienda?
Respuesta de referencia (<i>ground truth</i>)	No se ha dicho en ningún caso que las empresas en Andalucía deban garantizar los empleos y es el gobierno el que tiene la obligación de dar las posibilidades de acceso a la vivienda y no destruir empleos o empresas en Andalucía.
Campo	Valor
Archivo	DSCA080026_1.xml
Tipo de pregunta	Consecutive
Chunks relevantes	[3, 4, 5]
Pregunta	¿Cómo la propuesta del Partido Popular para garantizar más agua en Andalucía se relacionó con el trasvase del Ebro y cómo se benefició Málaga de esta acción?
Respuesta de referencia (<i>ground truth</i>)	La propuesta del Partido Popular para garantizar más agua en Andalucía se relacionó con el trasvase del Ebro ya que el trasvase del Ebro suponía para Almería 95 hectómetros cúbicos, y la propuesta del Partido Popular garantizó 189 hectómetros cúbicos. Además, el trasvase del Ebro se olvidaba de la afectada en Andalucía, por la sequía, que era Málaga. Sin embargo, Málaga no se benefició de esta acción ya que no se beneficiaba ni en una sola gota.
Campo	Valor
Archivo	DSCA080036_1.xml
Tipo de pregunta	Consecutive
Chunks relevantes	[6, 7, 8]
Pregunta	¿Cómo puede la simplificación, la homogeneidad y la normalización administrativa ayudar a garantizar la objetividad e imparcialidad en el tratamiento de asuntos de ciudadanos?
Respuesta de referencia (<i>ground truth</i>)	La simplificación, la homogeneidad y la normalización administrativa pueden ayudar a garantizar la objetividad e imparcialidad en el tratamiento de asuntos de ciudadanos al facilitar el acceso a información y servicios administrativos para todos los ciudadanos, independientemente de su lugar de residencia o condición socioeconómica. Esto puede ayudar a evitar discriminación y favoritismo en el proceso de resolución de asuntos y garantizar que todos los ciudadanos tengan el mismo derecho a ser atendidos e informados por la administración pública.
Campo	Valor
Archivo	DSCA080059_6.xml
Tipo de pregunta	Consecutive
Chunks relevantes	[19, 20, 21]
Pregunta	¿Cuántas intervenciones ha realizado el dispositivo Infoca en los últimos incendios, y qué provincias han sido afectadas con mayor superficie?
Respuesta de referencia (<i>ground truth</i>)	El dispositivo Infoca ha realizado un total de 12 intervenciones en los últimos incendios, con mayor superficie afectada en la provincia de Cádiz (985 hectáreas) y en la provincia de Almería (378 hectáreas).

Capítulo 6

Implementación

En este capítulo se describe la implementación de los distintos sistemas de recuperación aumentada (RAG) [24, 20] desarrollados en este trabajo. El objetivo principal es comparar diferentes estrategias de recuperación de información aplicadas al mismo conjunto de datos, con el fin de analizar su impacto en la calidad de las respuestas generadas.

En concreto, se han implementado cuatro enfoques distintos. En primer lugar, un sistema basado en BM25 [47], que utiliza un modelo clásico de recuperación léxica. En segundo lugar, un sistema basado en embeddings [21], que permite realizar recuperación semántica mediante representaciones vectoriales de los textos. En tercer lugar, un sistema híbrido [25] implementado sobre OpenSearch que combina señales léxicas y semánticas dentro del propio motor de búsqueda. Finalmente, se ha desarrollado un sistema híbrido basado en Reciprocal Rank Fusion (RRF) [7], que integra los resultados de los enfoques BM25 y embedding mediante una estrategia de fusión de rankings.

A lo largo del capítulo se detallan las principales decisiones de implementación, la arquitectura de cada sistema y su integración en el pipeline de generación de respuestas.

Además, con el objetivo de facilitar la reproducibilidad del trabajo, el código fuente completo de las implementaciones desarrolladas está disponible en el siguiente repositorio de GitHub: <https://github.com/joseroman0306/TFG-RAG>.

6.1. Componentes comunes

En esta sección se describen los componentes comunes a todos los sistemas de recuperación aumentada (RAG) implementados en este trabajo. Independientemente de la estrategia de recuperación utilizada —ya sea BM25, embeddings o enfoques híbridos— todos los sistemas comparten una misma estructura de pipeline que permite garantizar una comparación homogénea

entre métodos.

En concreto, se detallan las etapas fundamentales que forman parte de dicho pipeline: la carga del conjunto de datos de evaluación, la conexión con el motor de búsqueda, la construcción del contexto a partir de los fragmentos recuperados, la generación de respuestas mediante un modelo de lenguaje y, finalmente, el proceso de evaluación tanto del rendimiento de recuperación como de la calidad de las respuestas generadas.

6.1.1. Carga del Dataset y conexión con el motor de búsqueda

En esta fase inicial del sistema se realizan dos operaciones fundamentales: la conexión con el motor de búsqueda y la carga del conjunto de datos de evaluación. Estas operaciones son comunes a todos los enfoques RAG implementados, ya que proporcionan la infraestructura básica necesaria para ejecutar el pipeline completo.

Carga del Dataset

```
with open(DATASET_FILE, "r", encoding="utf-8") as f:
    dataset = json.load(f)
```

Estas líneas de código se encargan de cargar el conjunto de datos de evaluación en memoria a partir de un fichero en formato JSON. En primer lugar, la instrucción `open(DATASET_FILE, "r", encoding="utf-8")` abre el archivo especificado en modo lectura, asegurando la correcta interpretación de caracteres especiales mediante la codificación UTF-8. A continuación, `json.load(f)` parsea el contenido del fichero y lo convierte en una estructura de datos nativa de Python, típicamente una lista de diccionarios, donde cada elemento representa una instancia del dataset. De este modo, el conjunto de evaluación queda completamente disponible en memoria para su posterior procesamiento dentro del pipeline RAG.

Conexión con OpenSearch

```
client = OpenSearch(
    hosts=[{"host": OS_HOST, "port": OS_PORT}],
    http_auth=(OS_USER, OS_PASS),
    use_ssl=True,
    verify_certs=False
)
```

Este fragmento de código se encarga de inicializar la conexión con el motor de búsqueda OpenSearch [36], el cual se utiliza como base para la recuperación de documentos en los distintos sistemas RAG implementados.

En concreto, se crea un cliente de OpenSearch especificando los parámetros de conexión necesarios. En primer lugar, se define la dirección del servidor mediante el host y el puerto, lo que permite establecer la comunicación con la instancia donde se encuentra el índice de documentos. A continuación, se incluyen las credenciales de acceso (`http_auth`), que garantizan la autenticación del usuario dentro del sistema. Finalmente, se habilita el uso de SSL y se desactiva la verificación de certificados (`verify_certs=False`), una configuración habitual en entornos de desarrollo que permite simplificar la conexión.

Una vez inicializado, este cliente se utiliza posteriormente en la fase de recuperación de información para realizar consultas sobre el índice y obtener los fragmentos de texto más relevantes en función de cada consulta del usuario.

6.1.2. Carga del modelo y cuantización

En esta fase del sistema se realiza la carga y configuración del modelo generativo utilizado para la producción de respuestas dentro del pipeline RAG. Este componente es común a todos los enfoques implementados, ya que el modelo de lenguaje se mantiene constante a lo largo de los distintos experimentos, permitiendo así una comparación justa entre las diferentes estrategias de recuperación de información.

Carga del tokenizer

```
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
```

Primero se carga el tokenizer asociado al modelo de lenguaje. Su función es convertir el texto en tokens (y viceversa), es decir, transformar el lenguaje natural en una representación numérica que el modelo puede procesar.

Además algunos modelos no tienen definido un token de padding. En ese caso, se asigna el token de fin de secuencia (`eos_token`) como padding. Esto evita errores durante la tokenización de entradas con longitudes variables.

Configuración de cuantización (4-bit)

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True
)
```

Esta parte configura la carga del modelo en formato cuantizado a 4 bits, utilizando la librería BitsAndBytes [9].

- `load_in_4bit=True`: reduce el tamaño del modelo a 4 bits para ahorrar memoria.
- `bnb_4bit_compute_dtype=torch.float16`: los cálculos internos se realizan en precisión float16.
- `bnb_4bit_quant_type="nf4"`: usa cuantización tipo NormalFloat4, optimizada para modelos de lenguaje.
- `bnb_4bit_use_double_quant=True`: aplica doble cuantización para mejorar eficiencia sin perder demasiada calidad.

Esta es una configuración bastante común cuando se utiliza una cuantización 4-bits.

Además se define la distribución de memoria entre GPU y CPU.

```
max_memory = {
    0: "6GB",
    "cpu": "32GB"
}
```

Esto permite ejecutar modelos grandes aunque no quepan completamente en GPU.

Carga del modelo

```
model = AutoModelForCausalLM.from_pretrained(
    MODEL_NAME,
    device_map="balanced",
    max_memory=max_memory,
    quantization_config=bnb_config,
    dtype=torch.float16,
    low_cpu_mem_usage=True
)
```

Aquí se carga el modelo generativo de lenguaje.

Parámetros principales:

- `device_map="balanced"`: distribuye automáticamente el modelo entre CPU y GPU.
- `max_memory`: limita el uso de memoria por dispositivo.
- `quantization_config`: aplica la cuantización a 4 bits definida antes.

- `dtype=torch.float16`: usa media precisión para cálculos más eficientes.
- `low_cpu_mem_usage=True`: optimiza el uso de RAM durante la carga.

En conjunto, la carga del tokenizer, la configuración de la cuantización y la inicialización del modelo permiten ejecutar un modelo de lenguaje de gran tamaño de forma eficiente en el hardware disponible. El uso de cuantización a 4 bits y la distribución automática de memoria entre CPU y GPU reducen significativamente los requisitos computacionales, manteniendo al mismo tiempo un nivel de rendimiento adecuado para las tareas de generación de respuestas. Esta configuración común se emplea en todos los sistemas RAG implementados, garantizando que las diferencias observadas en los resultados experimentales se deban exclusivamente a las estrategias de recuperación utilizadas y no al modelo generativo.

6.1.3. Generación con el LLM

En esta fase se prepara la información que será utilizada por el modelo de lenguaje para generar una respuesta a partir de una consulta dada. Para ello, se construye una entrada estructurada que combina las instrucciones proporcionadas al modelo, la información contextual disponible y la pregunta del usuario. Esta etapa es común a todos los sistemas implementados, ya que independientemente del método de recuperación empleado, el proceso de generación de respuestas se realiza utilizando el mismo modelo de lenguaje y una estrategia de prompting equivalente.

Los fragmentos de texto proporcionados al modelo se agrupan en una única cadena de texto denominada contexto. Para ello, todos los elementos recuperados se concatenan utilizando saltos de línea dobles como separador.

```
context = "\n\n".join(chunks)
```

De esta forma, el modelo recibe una única entrada que contiene toda la información relevante disponible para responder a la consulta.

A continuación se construye el prompt que será enviado al modelo de lenguaje.

```
prompt = f"""
Responde la siguiente pregunta usando únicamente
el contexto proporcionado.
```

```
Si la respuesta no está completamente en el
contexto, responde parcialmente con lo que haya
disponible.
```

No inventes información.

CONTEXTO:
{context}

PREGUNTA:
{query}

RESPUESTA:
"""

El prompt incluye tres elementos principales:

- Instrucciones para restringir la generación a la información disponible.
- El contexto que contiene los fragmentos de texto.
- La pregunta formulada por el usuario.

Esta estrategia busca minimizar las alucinaciones del modelo y favorecer respuestas fundamentadas en la información proporcionada.

Una vez construido el prompt, se tokeniza y se envía al modelo para generar la respuesta.

```
inputs = tokenizer(  
    prompt,  
    return_tensors="pt",  
    truncation=True,  
    max_length=2048  
) .to(DEVICE)
```

La tokenización transforma el texto en una representación numérica adecuada para el modelo. Además, se establece una longitud máxima de 2048 tokens para evitar superar la ventana de contexto disponible.

Posteriormente se realiza la generación:

```
output = model.generate(  
    **inputs,  
    max_new_tokens=120,  
    do_sample=True,  
    temperature=0.7,  
    top_p=0.9,  
    pad_token_id=tokenizer.eos_token_id  
)
```

Los parámetros más relevantes son:

- `max_new_tokens`: limita la longitud máxima de la respuesta generada.
- `do_sample`: determina si la generación utiliza muestreo o selección determinista.
- `temperature`: controla el grado de aleatoriedad durante la generación.
- `top_p`: restringe la selección de tokens a los más probables según un umbral acumulado.

Los valores concretos de estos parámetros pueden variar en función de la configuración experimental utilizada. Su análisis y ajuste se describen posteriormente.

Finalmente, la secuencia generada se decodifica y se elimina el texto correspondiente al prompt original para obtener únicamente la respuesta producida por el modelo.

```
decoded = tokenizer.decode(  
    output[0],  
    skip_special_tokens=True  
)  
  
answer = decoded[len(prompt):].strip()
```

Todo el proceso descrito se implementa de forma unificada en la función `generate_answer(query, chunks)`, la cual es compartida por todos los sistemas RAG desarrollados. De este modo se asegura la consistencia del componente generativo, de manera que las diferencias observadas en los resultados experimentales se deban exclusivamente a la fase de recuperación de información.

6.1.4. Evaluación el sistema

En esta fase se describe el procedimiento seguido para evaluar el rendimiento de los distintos sistemas RAG implementados. El objetivo de esta fase es cuantificar tanto la calidad del proceso de recuperación de información como la calidad de las respuestas generadas por el modelo de lenguaje. Para ello, se definen métricas específicas que permiten analizar de forma separada el comportamiento del sistema en la etapa de retrieval y en la etapa de generación, facilitando así la comparación entre los diferentes enfoques propuestos.

Evaluación de Retrieval

La evaluación de la fase de recuperación se basa en la comparación entre los fragmentos relevantes definidos en el conjunto de datos de evaluación y los

fragmentos efectivamente recuperados por cada sistema. Para ello, se implementa una función de evaluación `evaluate_retrieval(sample, sources)` que opera a nivel de conjunto, permitiendo cuantificar el grado de coincidencia entre ambos.

En primer lugar, se extrae la información de referencia del conjunto de evaluación, concretamente los identificadores de los fragmentos relevantes asociados a la consulta y el documento de origen.

```
relevant_chunks = sample.get("relevant_chunks", [])
archivo = sample.get("archivo", "")
```

A continuación, estos identificadores se convierten a un conjunto de enteros. El uso de un conjunto elimina duplicados y facilita el cálculo de intersecciones posteriores.

```
relevant_set = set(int(chunk) for chunk in relevant_chunks)
```

Después se construye el conjunto de fragmentos recuperados por el sistema, filtrando únicamente aquellos que pertenecen al mismo documento (archivo) que la consulta original. Esto garantiza que la evaluación se realice de forma consistente a nivel de documento. Además, se convierte cada identificador a entero para asegurar compatibilidad con el formato del dataset.

```
retrieved_set = set(
    int(s["chunk_id"]) for s in sources
    if s["archivo"] == archivo and s.get("chunk_id") is not None
)
```

Una vez definidos ambos conjuntos, se calcula la intersección entre ellos, es decir, los fragmentos que han sido correctamente recuperados por el sistema.

```
intersection_len = len(relevant_set.intersection(retrieved_set))
```

A partir de esta intersección se obtienen las métricas principales de evaluación del retrieval [27]:

- **Hit Rate@K**: indica si al menos un fragmento relevante ha sido recuperado. Es una métrica binaria (1 si existe intersección, 0 en caso contrario).
- **Precision@K**: mide qué proporción de los fragmentos recuperados son realmente relevantes.
- **Recall@K**: mide qué proporción de los fragmentos relevantes han sido recuperados respecto al total esperado.

```
hit = 1 if intersection_len > 0 else 0
precision = intersection_len / TOP_K
```

El recall se calcula dividiendo los fragmentos correctamente recuperados entre el total de fragmentos relevantes disponibles. En caso de que no existan fragmentos relevantes definidos, se evita la división por cero devolviendo 0.

```
if len(relevant_set) > 0:
    recall = intersection_len / len(relevant_set)
else:
    recall = 0.0
```

Finalmente, la función devuelve las tres métricas, que serán agregadas posteriormente para obtener una evaluación global del sistema de recuperación.

Evaluación de Generación

En esta parte se describe la evaluación del componente generativo del sistema RAG, encargado de producir respuestas en lenguaje natural a partir de la información recuperada y la consulta del usuario. A diferencia de la evaluación del retrieval, en este caso el objetivo es analizar la calidad del texto generado por modelo de lenguaje, considerando tanto su adecuación a la pregunta como su fidelidad al contexto proporcionado. Para ello, se emplea una evaluación basada en métricas automáticas obtenidas mediante el propio modelo como evaluador, lo que permite estimar de forma consistente distintos aspectos de la calidad de la respuesta generada.

En primer lugar, se construye una entrada estructurada en forma de prompt que incluye la pregunta, el contexto utilizado para la generación y la respuesta producida por el modelo. Este formato permite a un modelo evaluador analizar de manera explícita la relación entre estos tres elementos.

```
context = "\n\n".join(chunks)
```

El contexto se construye concatenando los fragmentos recuperados en una única cadena de texto, separada por saltos de línea dobles, con el fin de mantener la coherencia estructural de la información.

A continuación se define el prompt de evaluación:

```
You are an evaluator of Retrieval-Augmented Generation (RAG) systems.
```

```
QUESTION:
{query}
```

```
CONTEXT:
```

```
{context}
```

```
GENERATED ANSWER:
{answer}
```

Score from 1 to 5:

1. context_relevance (Is the context useful for answering?)
2. faithfulness (Is the answer grounded in the context?)
3. answer_relevance (Does the answer respond to the question?)

Return ONLY valid JSON:

```
{{
"context_relevance": number,
"faithfulness": number,
"answer_relevance": number
}}
```

Este prompt instruye al modelo a actuar como evaluador de un sistema RAG, solicitándole que asigne una puntuación entre 1 y 5 a tres aspectos:

- context_relevance: utilidad del contexto para responder la pregunta.
- faithfulness: grado en el que la respuesta está respaldada por el contexto.
- answer_relevance: adecuación de la respuesta a la pregunta planteada.

Una vez construido el prompt, este se tokeniza y se envía al modelo evaluador:

Se solicita que la salida se devuelva en formato JSON estricto para facilitar su posterior procesamiento.

```
inputs = tokenizer(
    prompt,
    return_tensors="pt",
    truncation=True,
    max_length=2048
).to(DEVICE)
```

La tokenización transforma el texto en una representación numérica compatible con el modelo, limitando la entrada a un máximo de 2048 tokens para evitar exceder la ventana de contexto.

Posteriormente, el modelo genera la respuesta de evaluación:

```
output = model.generate(  
    **inputs,  
    max_new_tokens=120,  
    do_sample=False,  
    pad_token_id=tokenizer.eos_token_id  
)
```

En este caso se utiliza generación determinista (`do_sample=False`) para asegurar estabilidad en las métricas. El resultado se decodifica y se elimina el prompt original para obtener únicamente la respuesta generada.

```
decoded = tokenizer.decode(output[0], skip_special_tokens=True)  
response = decoded[len(prompt):].strip()
```

Dado que la salida puede contener ruido o formato incorrecto, se aplica un proceso de limpieza y extracción mediante expresiones regulares:

```
match = re.search(r"\{.*?\}", response, re.DOTALL)
```

Esto permite localizar el bloque JSON dentro de la respuesta del modelo. Posteriormente, se parsea el JSON y se extraen las tres métricas [24].

```
metrics = json.loads(match.group())  
  
context_score = int(metrics.get("context_relevance", 0))  
faith_score = int(metrics.get("faithfulness", 0))  
answer_score = int(metrics.get("answer_relevance", 0))  
  
context_score = max(1, min(context_score, 5))  
faith_score = max(1, min(faith_score, 5))  
answer_score = max(1, min(answer_score, 5))
```

Finalmente, se realiza una normalización de los valores para garantizar que se encuentren dentro del rango esperado [1, 5]. En caso de error en el parseo o ausencia de respuesta válida, se devuelve un valor por defecto de (1, 1, 1).

6.2. Sistemas de recuperación

En esta sección se describen los distintos sistemas de recuperación implementados en este trabajo, que constituyen el núcleo comparativo del estudio. A diferencia de los componentes comunes presentados anteriormente, aquí se detallan las estrategias específicas empleadas para la recuperación de información, las cuales determinan en gran medida la calidad final de las respuestas generadas por el sistema RAG.

En concreto, se presentan tres enfoques principales: un sistema basado en BM25 como método de recuperación léxica, un sistema basado en embeddings que permite realizar búsqueda semántica mediante representaciones vectoriales, y finalmente distintas variantes híbridas que combinan ambas aproximaciones con el objetivo de mejorar la robustez y precisión del proceso de recuperación.

6.2.1. BM25

El primer sistema implementado utiliza BM25 [47] como mecanismo de recuperación de información. Este enfoque representa la aproximación clásica de búsqueda léxica, en la que la relevancia de un documento se determina a partir de la coincidencia de términos entre la consulta y los textos almacenados en el índice. Los fragmentos recuperados mediante este método son posteriormente utilizados para construir el contexto que será proporcionado al modelo de lenguaje durante la generación de respuestas.

La función `retrieve_chunks(query, top_k=TOP_K)` implementa la fase de recuperación de información utilizando BM25 a través de OpenSearch. Su objetivo es recibir una consulta en lenguaje natural, recuperar los fragmentos más relevantes del índice y devolver tanto el contenido textual como la información necesaria para la evaluación posterior.

En primer lugar, se construye la consulta que será enviada a OpenSearch:

```
body = {
    "size": top_k,
    "query": {
        "match": {
            "texto": {
                "query": query
            }
        }
    }
}
```

La consulta utiliza el operador `match` sobre el campo `texto` del índice. Este tipo de búsqueda activa el mecanismo de recuperación léxica de OpenSearch basado en BM25, calculando una puntuación de relevancia para cada documento según la coincidencia de términos entre la consulta y el texto almacenado.

El parámetro `size` limita el número de resultados devueltos a los `top_k` documentos más relevantes.

A continuación, la consulta se ejecuta sobre el índice:

```
resp = client.search(index=INDEX_NAME, body=body)
```

OpenSearch devuelve los documentos ordenados según la puntuación BM25 obtenida para cada uno de ellos.

Posteriormente se inicializan dos listas:

```
chunks = []
sources = []
```

La lista `chunks` almacenará el contenido textual que será utilizado posteriormente para construir el contexto enviado al modelo de lenguaje, mientras que `sources` almacenará información auxiliar empleada durante la evaluación del sistema.

Después se recorren todos los resultados recuperados:

```
for hit in resp["hits"]["hits"]:
```

Para cada resultado se extraen dos elementos fundamentales:

```
src = hit["_source"]
score = hit["_score"]
```

`src` contiene todos los campos originales almacenados en el documento.

`score` contiene la puntuación BM25 asignada por OpenSearch.

A continuación se construye un bloque de texto enriquecido:

```
bloque = f"""
Legislatura: ...
Órgano: ...
Presidente: ...
...
Texto:
{src.get('texto')}
""".strip()
```

Este bloque no incluye únicamente el contenido textual del fragmento, sino también información contextual adicional como la legislatura, el órgano parlamentario, la fecha, el tipo de sesión o el interviniente. De esta forma, el modelo generativo recibe más información para interpretar correctamente el contenido recuperado.

Cada bloque se añade a la lista de fragmentos y además, se almacena información de trazabilidad para la evaluación:

```
chunks.append(bloque)
sources.append({
    "archivo": src.get("archivo"),
    "chunk_id": src.get("chunk_id"),
    "score": score
})
```

Concretamente se guarda:

- El documento original (archivo).
- El identificador del fragmento (`chunk_id`).
- La puntuación BM25 (`score`).

Esta información se utilizará posteriormente para calcular métricas de recuperación como Hit Rate, Precision@K y Recall@K.

Finalmente, la función devuelve ambas estructuras.

Aunque la consulta no especifica explícitamente el algoritmo de ranking, OpenSearch utiliza BM25 como función de similitud por defecto para los campos de texto. Por tanto, la consulta `match` empleada en este trabajo realiza una recuperación léxica basada en BM25, asignando a cada fragmento una puntuación de relevancia (`_score`) que determina su posición en el ranking de resultados.

6.2.2. Embeddings

A diferencia del sistema BM25, donde la recuperación se basa en coincidencias léxicas entre términos, aquí la búsqueda se realiza utilizando representaciones vectoriales de los textos [21], lo que permite recuperar documentos con significado similar aunque no compartan exactamente las mismas palabras.

En primer lugar, la consulta del usuario se transforma en un embedding mediante la función `embed_query()`:

```
query_vector = embed_query(query)
```

Esta función genera una representación vectorial densa de la consulta. El embedding captura información semántica del texto, permitiendo comparar significados en lugar de únicamente palabras coincidentes.

A continuación se construye la consulta que será enviada a OpenSearch:

```
body = {
  "size": TOP_K,
  "query": {
    "knn": {
      "embedding": {
        "vector": query_vector,
        "k": TOP_K
      }
    }
  }
}
```

En este caso se utiliza una consulta de tipo knn (k-Nearest Neighbors), que busca los vectores más cercanos al embedding de la consulta dentro del espacio vectorial almacenado en el índice.

Los parámetros principales son:

- vector: embedding generado a partir de la consulta.
- k: número de vecinos más cercanos que se desean recuperar.
- size: número de resultados devueltos.

De esta forma, OpenSearch recupera los fragmentos cuyos embeddings presentan una mayor similitud con el embedding de la consulta.

Una vez definida la consulta, esta se ejecuta sobre el índice:

```
resp = client.search(index=INDEX_NAME, body=body)
```

Una vez recuperados los resultados, el procesamiento posterior es idéntico al descrito para el sistema BM25. Para cada fragmento recuperado se extrae su contenido y los metadatos asociados, construyéndose un bloque de texto enriquecido que posteriormente formará parte del contexto proporcionado al modelo de lenguaje. Asimismo, se almacena información auxiliar (*archivo* y *chunk_id*) utilizada durante la evaluación del sistema. Finalmente, la función devuelve las listas **chunks** y **sources**, que serán empleadas respectivamente en las fases de generación y evaluación.

6.2.3. Híbridos

Aquí se presentan los dos enfoques híbridos [25, 7] implementados en el sistema de recuperación. Ambos comparten el objetivo de combinar señales léxicas y semánticas para mejorar la calidad de los resultados, pero difieren en la forma en la que se realiza dicha combinación.

Por un lado, el enfoque híbrido basado en OpenSearch integra directamente BM25 y búsqueda por embeddings dentro del propio motor de búsqueda, delegando en el sistema la fusión de ambas señales en un único ranking. Por otro lado, el enfoque basado en Reciprocal Rank Fusion (RRF) realiza la combinación a nivel de aplicación, fusionando de forma externa los rankings obtenidos por BM25 y por embeddings mediante una estrategia de agregación de posiciones.

Híbrido de OpenSearch

En primer lugar, se genera la representación vectorial de la consulta:

```
query_vector = embed_query(query)
```

Este embedding captura el significado semántico de la consulta y será utilizado en la parte de búsqueda basada en k-NN.

A continuación, se construye una consulta híbrida en OpenSearch:

```
body = {
  "size": TOP_K,
  "query": {
    "hybrid": {
      "queries": [
        {
          "match": {
            "texto": {
              "query": query
            }
          }
        },
        {
          "knn": {
            "embedding": {
              "vector": query_vector,
              "k": 20
            }
          }
        }
      ]
    }
  }
}
```

Esta consulta combina dos estrategias:

- BM25 (match): realiza una búsqueda léxica basada en coincidencias de términos en el campo texto.
- k-NN sobre embeddings: recupera los documentos más cercanos semánticamente al embedding de la consulta.

El parámetro **k=20** en la parte vectorial permite recuperar un conjunto amplio de candidatos semánticos, que posteriormente se combinan con los resultados léxicos dentro del motor de búsqueda.

Una vez ejecutada la consulta:

```
resp = client.search(index=INDEX_NAME, body=body)
```

OpenSearch se encarga de fusionar ambas señales de relevancia y devolver un ranking unificado de documentos.

El resto del proceso de tratamiento de resultados es equivalente al descrito en el sistema BM25. Cada resultado se transforma en un bloque de contexto enriquecido con metadatos y se almacenan los identificadores necesarios para la evaluación posterior:

```
chunks = []
sources = []

for hit in resp["hits"]["hits"]:
    src = hit["_source"]

    bloque = f"""
    ...
    """.strip()

    chunks.append(bloque)

    sources.append({
        "archivo": src.get("archivo"),
        "chunk_id": src.get("chunk_id")
    })
```

Finalmente, la función devuelve:

- `chunks`: fragmentos que formarán el contexto del modelo de lenguaje.
- `sources`: información auxiliar para el cálculo de métricas de evaluación.

Reciprocal Rank Fusion

Este enfoque híbrido implementa la estrategia de Reciprocal Rank Fusion (RRF), que combina de forma externa los resultados obtenidos por dos sistemas de recuperación independientes: BM25 y búsqueda basada en embeddings (k-NN). A diferencia del enfoque híbrido integrado en OpenSearch, en este caso la fusión no se realiza dentro del motor de búsqueda, sino a nivel de aplicación, permitiendo un control explícito sobre cómo se combinan los rankings.

En primer lugar, se definen dos funciones de recuperación independientes. La primera implementa BM25 mediante una consulta `match` sobre el campo de texto:

```
def retrieve_bm25(query, size=20):
```

La segunda realiza recuperación semántica mediante embeddings y búsqueda k-NN:

```
def retrieve_knn(query, size=20):
```

Ambas funciones devuelven una lista de documentos ordenados según su relevancia en cada uno de los enfoques.

Posteriormente, estos dos rankings se combinan mediante la función de fusión RRF:

```
def reciprocal_rank_fusion(bm25_hits, knn_hits):
```

El proceso comienza inicializando estructuras auxiliares para almacenar puntuaciones, metadatos y contenido textual:

```
scores = {}
doc_info = {}
doc_text = {}
```

A continuación, se recorren todos los documentos recuperados por ambos sistemas para construir una representación unificada:

- doc_info almacena los metadatos necesarios para la evaluación (archivo y chunk_id).
- doc_text guarda el bloque de texto enriquecido que será utilizado como contexto en la generación.

Una vez preparadas las estructuras, se aplica el algoritmo RRF. Este método asigna a cada documento una puntuación basada en su posición en cada ranking individual, en lugar de su score absoluto:

```
scores[doc_id] += 1 / (RRF_K + rank + 1)
```

Este cálculo se realiza tanto para los resultados de BM25 como para los de k-NN, de forma independiente. De este modo, los documentos que aparecen en buenas posiciones en cualquiera de los dos rankings obtienen mayor puntuación final.

Después de acumular las contribuciones de ambos sistemas, los documentos se ordenan según su puntuación final:

```
ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
```

Finalmente, se seleccionan los TOP.K documentos más relevantes, construyendo dos estructuras de salida:

- retrieved: metadatos de evaluación (archivo y chunk_id).
- chunks: textos enriquecidos que formarán el contexto del modelo de lenguaje.

6.3. Uso del sistema

Con el objetivo de ilustrar el funcionamiento completo del pipeline RAG implementado, en esta sección se presenta un ejemplo representativo de ejecución del sistema. Este ejemplo permite observar de forma integrada las distintas fases descritas anteriormente: formulación de la consulta, recuperación de información, generación de la respuesta y trazabilidad de las fuentes utilizadas.

A diferencia de una ejecución aislada, este tipo de interacción resume el comportamiento real del sistema en un escenario de uso, en el que la respuesta final se construye a partir de fragmentos recuperados dinámicamente del índice documental.

Consulta del usuario: *¿Qué se habla el día 29 de abril de 2008 sobre el Consejo Asesor de RTVE en Andalucía?*

Fase de recuperación: Recuperando fragmentos relevantes mediante el sistema de búsqueda implementado (BM25).

Fase de generación: El modelo de lenguaje genera una respuesta fundamentada en el contexto recuperado.

Respuesta generada: La Mesa del Parlamento de Andalucía conoció las propuestas de designación de miembros del Consejo Asesor de la Radio y Televisión Española en Andalucía presentadas por los Grupos Parlamentarios Socialista, Popular de Andalucía e Izquierda Unida Los Verdes-Convocatoria por Andalucía. En la mañana del mismo día se realizó un escrutinio de los votos y se designó a todos los miembros propuestos miembros del Consejo Asesor de RTVE en Andalucía, excepto a doña María Antonia Morillas González, que recibió 106 votos.

Fuentes recuperadas:

■ Fuente 1

- Archivo: DSPA080004_5.xml
- Chunk ID: 0
- Legislatura: VIII
- Fecha: 29 abril 2008
- Interviniente: La señora DEL POZO FERNÁNDEZ
- Tipo de iniciativa: Consejo Asesor de RTVE en Andalucía
- Score OpenSearch: 14.014557

■ Fuente 2

- Archivo: DSPA080004_5.xml

- Chunk ID: 0
- Legislatura: VIII
- Fecha: 29 abril 2008
- Interviniente: La señora COVES BOTELLA
- Tipo de iniciativa: Consejo Asesor de RTVE en Andalucía
- Score: 11.99

■ **Fuente 3**

- Archivo: DSCA080236_1.xml
- Chunk ID: 1
- Legislatura: VIII
- Fecha: 7 octubre 2009
- Interviniente: La señora COVES BOTELLA
- Tipo iniciativa: Presupuestos del Parlamento
- Score: 11.98

Este ejemplo ilustra de forma integrada el funcionamiento del sistema RAG desarrollado, desde la formulación de la consulta hasta la generación de la respuesta final a partir de los fragmentos recuperados. Asimismo, permite observar cómo el sistema mantiene la trazabilidad completa de las fuentes utilizadas, lo que facilita la interpretación de los resultados y la validación de la información generada. En conjunto, este tipo de ejecución representa el comportamiento esperado del sistema en un entorno real de uso, donde la calidad de la respuesta depende directamente de la capacidad de recuperación de los distintos enfoques implementados.

Capítulo 7

Evaluación experimental y resultados

En este capítulo se presenta la evaluación experimental realizada sobre el sistema RAG [20, 24] desarrollado. El objetivo principal de esta fase es analizar el impacto de los distintos componentes y parámetros que intervienen en el proceso de recuperación y generación de respuestas, así como determinar la configuración que ofrece el mejor equilibrio entre precisión, relevancia y calidad de las respuestas generadas.

Para ello, se ha diseñado una serie de experimentos orientados a estudiar, en primer lugar, el comportamiento del sistema de recuperación de información. Dado que el índice documental ha sido construido utilizando fragmentos de tamaño fijo, se analiza la influencia del número de chunks recuperados sobre la calidad de las respuestas generadas y sobre las métricas de recuperación empleadas. Posteriormente, se estudia el impacto de distintos parámetros del modelo generativo sobre el rendimiento global del sistema.

Además, se realiza una comparación entre las respuestas generadas por un modelo de lenguaje utilizado de forma aislada y aquellas obtenidas mediante la arquitectura RAG, con el fin de evaluar la contribución real del mecanismo de recuperación de contexto. Complementariamente, se emplea una estrategia de evaluación basada en LLM-as-a-Judge [53, 13] para obtener una valoración automática de la calidad de las respuestas generadas.

Finalmente, se presenta una discusión conjunta de los resultados obtenidos, identificando las configuraciones más adecuadas y extrayendo conclusiones sobre el rendimiento global del sistema propuesto.

7.1. Experimentación con en número de chunks recuperados

En esta sección se analiza el impacto del número de chunks recuperados sobre el rendimiento del sistema RAG. El parámetro k , que determina la cantidad de fragmentos de información incorporados al contexto del modelo generativo, es un factor clave en el equilibrio entre la relevancia de la información proporcionada y la posible introducción de ruido.

Para estudiar este efecto, se mantiene fija la configuración del sistema de recuperación y del modelo generativo, variando únicamente el valor de k . De este modo, se busca aislar su influencia en la calidad de las respuestas generadas y en las métricas de evaluación asociadas.

Objetivo del experimento

El objetivo del experimento es analizar el comportamiento del sistema RAG ante variaciones en el número de chunks recuperados (k) utilizando un enfoque cualitativo. En lugar de emplear métricas de evaluación automáticas, se estudia cómo cambia la respuesta del sistema cuando se modifica la cantidad de información contextual proporcionada al modelo generativo.

Para ello, se selecciona una pregunta concreta representativa del conjunto de evaluación y se ejecuta el sistema bajo diferentes valores de k , manteniendo constante el resto de la configuración. El objetivo es observar de manera controlada el efecto que tiene la ampliación o reducción del contexto recuperado en la respuesta final generada.

Configuración experimental

En este experimento se utiliza un sistema de recuperación implementado sobre OpenSearch, empleando RRF como función de ranking para obtener los fragmentos de texto más relevantes en función de la consulta de entrada.

El modelo generativo utilizado es *Mistral-7B-Instruct-v0.1* [17] cuantizado a 4 bits, que se encarga de generar la respuesta final a partir del contexto recuperado y la pregunta.

El prompt instruye al modelo a responder de forma clara y precisa utilizando únicamente la información proporcionada en el contexto.

Los parámetros de generación del modelo se mantienen fijos durante todo el experimento y son los siguientes: se establece una longitud máxima de 256 tokens de salida (`max_new_tokens`), una temperatura de 0.5 para controlar el nivel de aleatoriedad en la generación y un valor de top-p de 0.9 para el muestreo por núcleo de probabilidad. Además, se utiliza `sampling` activado (`do_sample = True`) y el token de fin de secuencia como pad token.

De este modo, se garantiza que las variaciones observadas en las respuestas sean consecuencia exclusiva de los cambios en el número de chunks

recuperados (k), manteniendo constante el resto de componentes del sistema.

Procedimiento

El proceso experimental sigue un flujo secuencial común para todas las consultas evaluadas, con el objetivo de garantizar la consistencia entre ejecuciones y permitir el análisis del impacto del número de chunks recuperados.

En primer lugar, se toma una pregunta del conjunto de evaluación definida previamente. A modo de ejemplo, entre las consultas utilizadas se encuentran:

“¿Cómo se está movilizand o el sector de la construcción de viviendas protegidas, y qué progreso se ha hecho en el Plan de Vivienda y Suelo?”

“¿Cuál es la propuesta de la ley de protección de los animales en Andalucía?”

Estas preguntas se utilizan como entrada del sistema de recuperación, RRF, que devuelve los k fragmentos de texto más relevantes según cada consulta.

A continuación, los chunks recuperados se concatenan en un único bloque de contexto, el cual se inserta dentro de un prompt estructurado junto con la pregunta original. Este prompt se utiliza como entrada del modelo generativo.

Posteriormente, el modelo *Mistral-7B-Instruct-v0.1* genera una respuesta utilizando el contexto proporcionado y los parámetros de generación definidos en la configuración experimental.

Finalmente, la respuesta generada se almacena junto con la pregunta, el valor de k utilizado y los documentos recuperados, con el fin de permitir su análisis posterior en las fases de evaluación y comparación.

Resultados

En esta sección se presentan los resultados obtenidos al variar el número de chunks recuperados (k) en el sistema RAG, manteniendo fija el resto de la configuración experimental. Para ello, se seleccionan dos preguntas representativas del conjunto de evaluación, correspondientes a distintos dominios temáticos, con el objetivo de analizar el comportamiento del modelo generativo ante variaciones en la cantidad de contexto proporcionado. En cada caso, se muestran las respuestas generadas por el sistema para distintos valores de k , permitiendo observar de forma directa cómo la ampliación del contexto influye en la completitud, coherencia y nivel de detalle de las respuestas.

Respuesta con un chunks

¿Cómo se está movilizand o el sector de la construcción de viviendas protegidas, y qué progreso se ha hecho en el Plan de Vivienda y Suelo?

El sector de la construcción de viviendas protegidas está movilizand o mucho más que antes, lo que se ha hecho es que las empresas privadas están acometiendo proyectos de viviendas protegidas en mucho mayor número que lo hacían en 2007. En cuanto a las actuaciones de rehabilitación, se han concedido ya subvenciones para 65,5 % de las 28.600 actuaciones previstas en el plan para este año 2008.

¿Cuál es la propuesta de la ley de protección de los animales en Andalucía?

La propuesta de ley de protección de los animales en Andalucía es una propuesta de ley que, como señora AGUILERA GARCÍA, CONSEJERA DE GOBERNACIÓN, explica, tiene como objetivo la regulación de las condiciones de protección y bienestar de los que viven bajo posesión de las personas, y, en particular, de los animales de compañía que viven en el territorio de nuestra Comunidad Autónoma. Esta propuesta de ley se aprobó en la pasada legislatura y tiene el objeto de garantizar la protección de los animales de compañía y la defensa de sus derechos, así como la gestión que tienen que realizar los ayuntamientos de Andalucía en esta cuestión.

Respuesta con tres chunks

¿Cómo se está movilizand o el sector de la construcción de viviendas protegidas, y qué progreso se ha hecho en el Plan de Vivienda y Suelo?

El sector de la construcción de viviendas protegidas se está movilizand o mucho más que antes, ya que las empresas privadas están acometiendo mucho más proyectos de viviendas protegidas que en 2007. En cuanto a las actuaciones de rehabilitación, se han concedido ya subvenciones para 65,5 % de las 28.600 actuaciones previstas en el plan para este año 2008.

La construcción de viviendas protegidas en alquiler está en un nivel del 65,5 %, y se ha progresado en casi cinco puntos desde el cuarto plan hasta el quinto plan. La participación del sector privado en la construcción de VPO ha aumentado en el régimen de alquiler.

¿Cuál es la propuesta de la ley de protección de los animales en Andalucía?

La propuesta de ley de protección de los animales en Andalucía es una de las propuestas de ley de protección de los animales que presentamos en este Parlamento. El Gobierno de Andalucía ha promulgado la Ley 11/2003, de Protección de Animales, cuyo objeto es la regulación de las condiciones de protección y bienestar de los que viven bajo posesión de las personas,

y, en particular, de los animales de compañía que viven en el territorio de nuestra Comunidad Autónoma. En la pasada legislatura tuvo lugar la aprobación del Decreto 42/2008, de 26 de septiembre, por el que se estableció la obligatoriedad de implantar sistemas de control de animales de compañía en el territorio de Andalucía.

Respuesta con cinco chunks

¿Cómo se está movilizando el sector de la construcción de viviendas protegidas, y qué progreso se ha hecho en el Plan de Vivienda y Suelo?

En este momento se está movilizando el sector de la construcción de viviendas protegidas en un nivel del 65,5 % de los planes previstos en el Plan de Vivienda y Suelo para el año 2008. La construcción de viviendas protegidas en alquiler de renta básica y viviendas protegidas para la integración social, en particular, ha progresado mucho. En el caso de las actuaciones de rehabilitación, a fecha de 25 de agosto se han concedido ya subvenciones para 18.727 actuaciones. En cuanto a las actuaciones en materia de construcción de viviendas protegidas, el sector está movilizando un poco más que lo que antes no le era de interés, es decir, que es la construcción de viviendas protegidas en alquiler.

¿Cuál es la propuesta de la ley de protección de los animales en Andalucía?

La propuesta de ley de protección de los animales en Andalucía es la Ley de Protección de Animales, aprobada en el año 2003. La Ley 11/2003, de Protección de Animales, cuyo objeto es la regulación de las condiciones de protección y bienestar de los que viven bajo posesión de las personas, y, en particular, de los animales de compañía que viven en el territorio de nuestra Comunidad Autónoma. En la pasada legislatura tuvo lugar la aprobación del Decreto 42/2008, de 16 de noviembre, que modifica el artículo 10 de la Ley 11/2003.

Respuesta con diez chunks

¿Cómo se está movilizando el sector de la construcción de viviendas protegidas, y qué progreso se ha hecho en el Plan de Vivienda y Suelo?

En cuanto a la construcción de viviendas protegidas, se ha movilizado el sector de la construcción en dos sentidos. Por un lado, se ha intentado atraer a este sector, que se considera una de las principales actividades económicas de la región, hacia la construcción de viviendas protegidas. Esto se ha hecho mediante la aprobación de diversos planes, como el Plan Andaluz de Vivienda y Suelo de 1992, o el Plan de Vivienda y Suelo de 2000, o el Plan Concertado de Vivienda y Suelo de 2008-2012, y también mediante

el establecimiento de diversas medidas de incentivo y de garantía para el sector de la construcción, como las ayudas de financiación de las empresas de la construcción de viviendas protegidas, o las garantías de crédito de las empresas de la construcción de viviendas protegidas.

¿Cuál es la propuesta de la ley de protección de los animales en Andalucía?

La propuesta de ley de protección de los animales en Andalucía, que fue aprobada en noviembre de 2003, es una ley de protección de animales de las que se derivan las siguientes disposiciones:

1. La ley 50/1999, de Tenencia de Animales Potencialmente Peligrosos, de 1999, establece los requisitos que deben cumplir los animales de esta categoría para su tenencia.

2. La ley 51/1999, de Protección de Animales en Vías Públicas, de 1999, establece los requisitos para la tenencia de animales en vías públicas.

3. La ley 52/1999, de Protección de Animales en Zonas Rurales y Pesquerías, de 1999, establece los requisitos para la tenencia de animales en zonas rurales y pesquerías.

Influencia de k en la completitud de las respuestas

Los resultados obtenidos muestran que el número de chunks recuperados influye directamente en la cantidad de información que el modelo es capaz de incorporar a la respuesta final. En general, a medida que aumenta el valor de k, el sistema dispone de un contexto más amplio, lo que favorece la inclusión de detalles adicionales.

Este comportamiento puede observarse en la pregunta relacionada con el sector de la construcción de viviendas protegidas. Con $k=1$, la respuesta contiene la información relativa al aumento de la participación de las empresas privadas y a las subvenciones concedidas para actuaciones de rehabilitación. Al incrementar el número de chunks a $k=3$, la respuesta incorpora información adicional sobre la construcción de viviendas protegidas en régimen de alquiler y la evolución experimentada entre distintos planes de vivienda, proporcionando una visión más completa del tema. Con $k=5$, el modelo añade nuevos detalles relacionados con las actuaciones realizadas y con diferentes modalidades de vivienda protegida, enriqueciendo aún más la respuesta.

De forma similar, en la pregunta sobre la ley de protección de los animales en Andalucía, la respuesta generada con $k=1$ recoge el objetivo general de la normativa, mientras que con $k=3$ y $k=5$ se añaden referencias específicas a la Ley 11/2003 y al Decreto 42/2008, aportando un mayor nivel de detalle y precisión normativa.

Sin embargo, el incremento de k hasta valores más elevados, como $k=10$, no produce una mejora equivalente en la completitud de las respuestas. Aunque el modelo dispone de una mayor cantidad de contexto, la información adicional incorporada no siempre responde de forma directa a la pregunta

planteada. En lugar de ampliar los aspectos principales de la respuesta, el modelo tiende a incluir información complementaria o contextual que no resulta estrictamente necesaria para responder a la consulta.

En conjunto, los resultados muestran que el aumento del número de chunks mejora la completitud de las respuestas hasta valores intermedios de k , a partir de los cuales los beneficios obtenidos son cada vez menores.

Influencia de k en el tiempo de ejecución

Además de afectar al contenido de las respuestas generadas, el número de *chunks* recuperados tiene un impacto directo sobre el tiempo necesario para completar el proceso de generación. A medida que aumenta el valor de k , el sistema debe recuperar una mayor cantidad de contexto e incorporarlo al *prompt* enviado al modelo generativo. Como consecuencia, aumenta el número de tokens procesados y el tiempo requerido para generar la respuesta final.

Número de chunks (k)	Tiempo de ejecución
1	~ 30 s
3	~ 2 min
5	~ 5 min
10	10–12 min

Cuadro 7.1: Tiempo aproximado de generación en función del número de chunks recuperados.

Como puede observarse en la Tabla 7.1, el tiempo de ejecución aumenta de forma significativa conforme se incrementa el número de *chunks* recuperados. Mientras que con $k = 1$ la respuesta se genera en aproximadamente 30 segundos, al utilizar $k = 10$ el tiempo de generación supera los 10 minutos.

Este comportamiento se debe principalmente al aumento del tamaño del contexto proporcionado al modelo. Cada *chunk* adicional incrementa el número de tokens incluidos en el *prompt*, lo que implica una mayor carga computacional durante la fase de inferencia. Además, al utilizar un modelo de 7 mil millones de parámetros ejecutado localmente, el impacto del tamaño del contexto resulta especialmente notable.

Por tanto, aunque valores elevados de k pueden proporcionar más información al modelo, también incrementan considerablemente el coste computacional. Esto pone de manifiesto la necesidad de encontrar un equilibrio entre la cantidad de contexto recuperado y el tiempo de respuesta del sistema.

Valor óptimo de k

A partir de los resultados obtenidos en los apartados anteriores, se puede analizar el compromiso existente entre la calidad de las respuestas generadas y el coste computacional asociado al número de *chunks* recuperados. Por un lado, el incremento de k mejora la completitud de las respuestas, especialmente al pasar de valores bajos (por ejemplo, $k = 1$) a valores intermedios ($k = 3$ y $k = 5$), donde el modelo es capaz de incorporar información más relevante y detallada.

Sin embargo, este aumento en la cantidad de contexto no se traduce en una mejora proporcional de la eficiencia del sistema. Tal y como se ha observado, el tiempo de generación crece de forma significativa a medida que aumenta k , alcanzando valores considerablemente elevados para $k = 10$, sin que ello implique una mejora clara en la calidad de las respuestas.

En este sentido, se identifica que valores intermedios de k , concretamente $k = 3$ y $k = 5$, ofrecen el mejor equilibrio entre calidad de respuesta y coste computacional. Estos valores permiten al sistema recuperar suficiente información relevante para construir respuestas completas, sin introducir una sobrecarga excesiva de contexto ni un incremento desproporcionado del tiempo de inferencia.

Por tanto, se concluye que el valor óptimo de k en este sistema RAG se sitúa en el rango intermedio, siendo $k = 3$ una configuración especialmente eficiente en términos de tiempo, mientras que $k = 5$ proporciona un mayor nivel de detalle a costa de un incremento moderado del coste computacional.

7.2. Resultados de métricas de retrieval

En esta sección se presentan los resultados obtenidos en la evaluación del sistema de recuperación de información. La evaluación se realiza sobre el conjunto de 40 preguntas pertenecientes al dataset de evaluación.

Para el análisis se consideran dos configuraciones del sistema en función del número de *chunks* recuperados, concretamente $k = 3$ y $k = 5$. Sobre estas configuraciones se calculan métricas estándar de Information Retrieval [27], como *Hit Rate*, *Precision* y *Recall* a nivel de los k documentos recuperados. Estas métricas permiten analizar la capacidad del sistema para recuperar información relevante en función del tamaño del contexto utilizado.

Sistema	HitRate@3	Precision@3	Recall@3
BM25	0.700	0.267	0.388
Embeddings	0.625	0.250	0.333
Híbrido OpenSearch	0.575	0.192	0.283
Híbrido RRF	0.725	0.292	0.396

Cuadro 7.2: Comparación de métricas de retrieval para distintos sistemas con $k = 3$ sobre el conjunto de 40 preguntas de evaluación.

Sistema	HitRate@5	Precision@5	Recall@5
BM25	0.775	0.190	0.446
Embeddings	0.725	0.175	0.412
Híbrido OpenSearch	0.700	0.267	0.387
Híbrido RRF	0.775	0.200	0.458

Cuadro 7.3: Comparación de métricas de retrieval para distintos sistemas con $k = 5$ sobre el conjunto de 40 preguntas de evaluación.

A partir de los resultados obtenidos, se pueden extraer varias conclusiones relevantes sobre el comportamiento de los distintos sistemas de recuperación evaluados.

En primer lugar, el sistema híbrido basado en RRF (*Reciprocal Rank Fusion*) [7] se consolida como la opción más sólida y consistente en términos de cobertura. En la configuración $k = 3$, este enfoque supera al resto de alternativas en todas las métricas, alcanzando un *Hit Rate* de 0.725, una *Precision* de 0.292 y un *Recall* de 0.396. Al incrementar el parámetro a $k = 5$, el Híbrido RRF mantiene el liderazgo absoluto en *Recall* (0.458) y comparte el máximo valor de *Hit Rate* (0.775) junto con el algoritmo tradicional BM25. Estos resultados demuestran que la fusión ponderada de rankings es una estrategia altamente efectiva en este dominio, logrando mitigar las debilidades individuales de las aproximaciones léxicas y semánticas.

Por su parte, el método léxico BM25 muestra un rendimiento sorprendentemente competitivo, posicionándose firmemente como el segundo mejor sistema global. Con $k = 3$, sus métricas superan de manera clara tanto a los *embeddings* puros como al enfoque híbrido nativo de OpenSearch. Al pasar a $k = 5$, BM25 iguala el *Hit Rate* de RRF (0.775) y se sitúa inmediatamente detrás en *Recall* (0.446). No obstante, este comportamiento confirma la tendencia de BM25 a recuperar términos exactos de forma masiva, lo que resulta útil para localizar información clave pero introduce ruido en contextos más amplios.

En contraste, el comportamiento del sistema Híbrido de OpenSearch resulta anómalo y dependiente del valor de k . Con un contexto reducido ($k = 3$), este sistema registra el rendimiento más bajo de la comparativa en todas las métricas. Sin embargo, al ampliar la ventana a $k = 5$, experimenta

7.3. Experimentación con distintos valores de la temperatura del modelo generativo

una notable mejoría, alcanzando la mayor *Precision* de dicha configuración (0.267) y un *Recall* competitivo (0.387). Este fenómeno sugiere que la combinación lineal o la indexación híbrida por defecto de OpenSearch requiere de un mayor volumen de documentos recuperados para equilibrar adecuadamente las puntuaciones semánticas y léxicas.

Por último, el modelo basado exclusivamente en *embeddings* densos presenta un rendimiento modesto, superando únicamente al híbrido de OpenSearch en la configuración de $k = 3$. Su incapacidad para liderar ninguna métrica indica que la representación vectorial pura encuentra dificultades para capturar con precisión las particularidades del lenguaje parlamentario, el cual suele depender fuertemente de tecnicismos, nombres propios o referencias textuales explícitas que el emparejamiento por densidad semántica no logra resolver de manera aislada.

Respecto al efecto general del parámetro k , se aprecia con claridad el clásico compromiso (*trade-off*) entre exhaustividad y precisión en los sistemas de recuperación de información. Al aumentar el número de *chunks* de 3 a 5, todos los sistemas —con la única excepción del Híbrido de OpenSearch— experimentan un incremento en su *Recall* y su *Hit Rate* a costa de una degradación notable en su *Precision*. Esto confirma que ampliar el contexto facilita la captura de una mayor cantidad de fragmentos relevantes, pero introduce un volumen de ruido superior en los primeros puestos del ranking.

7.3. Experimentación con distintos valores de la temperatura del modelo generativo

En esta sección se analiza el impacto de la configuración de los parámetros de generación del modelo en la calidad de las respuestas producidas por el sistema RAG. A diferencia de los experimentos centrados en la fase de recuperación, en este caso se mantiene fijo el sistema de retrieval, basado en un enfoque híbrido implementado en OpenSearch, y se varían únicamente los hiperparámetros del modelo generativo *Mistral-7B-Instruct-v0.1* [17]. El objetivo es estudiar cómo cambios en aspectos como la temperatura afectan al grado de coherencia, precisión y estabilidad de las respuestas generadas a partir del mismo contexto recuperado.

Do_sample = False

¿Qué se dice sobre la agricultura sostenible?

La agricultura sostenible se refiere a una agricultura que se basa en el uso de métodos de producción que no dañen el medio ambiente y que no dejan de ser sostenibles en el largo plazo.

La agricultura sostenible se refiere a una agricultura que se basa en el

uso de métodos de producción que no dañen el medio ambiente y que no dejen de ser sostenibles en el largo plazo.

Variaciones en la temperatura

¿Qué se dice sobre la agricultura sostenible?

Temperatura=0.3

La agricultura sostenible es una de las materias que se han hablado en el debate sobre el estado de la Comunidad Autónoma. Se dice que Andalucía sostenible es una de las principales prioridades de la Junta de Andalucía. El presidente de la Junta de Andalucía, el señor Griñán Martínez, dice que Andalucía sostenible es una de las principales prioridades de la Junta de Andalucía.

La agricultura sostenible se refiere a una forma de agricultura que se basa en la conservación de los recursos naturales y en el mantenimiento de la biodiversidad, con el fin de garantizar la producción de alimentos de alta calidad y la conservación de la salud y el bienestar de la población. Esta forma de agricultura se ha desarrollado en respuesta a los problemas ambientales y sociales que se han ido generando en el mundo, como la pérdida de biodiversidad, el cambio climático y la contaminación del agua y del aire. La agricultura sostenible se basa en el uso de técnicas y prácticas agrícolas que son más eficientes y menos dañinas para el medio ambiente y para la salud y el bienestar de la población.

La agricultura sostenible es una de las materias que se discute en el debate sobre el estado de la Comunidad Autónoma. El señor GRINÁN MARTÍNEZ, presidente de la Junta de Andalucía, dice que no se ha hablado sobre la agricultura sostenible en el debate sobre el estado de la Comunidad Autónoma.

Temperatura=0.5

La agricultura sostenible se refiere a una agricultura que se basa en la conservación de los recursos naturales y en la sustentabilidad económica y social. En el texto se dice que la agricultura sostenible se refiere a la agricultura andaluza.

La agricultura sostenible es una actividad agrícola que se basa en el uso de métodos y técnicas que no afecten negativamente al medio ambiente y que garantizan la conservación de los recursos naturales. La agricultura sostenible es una forma de producción agrícola que se basa en el uso de métodos y técnicas que no afecten negativamente al medio ambiente y que garantizan la conservación de los recursos naturales.

La agricultura sostenible se refiere a una agricultura que se basa en el uso de métodos y técnicas que no dañen el medio ambiente y que no afecten negativamente a la salud humana y a la biodiversidad. La agricultura sostenible también se basa en la producción de productos de alta calidad y en la creación de empleos sostenibles.

Temperatura=0.7

En el texto se dice que la actividad agrícola, la parte agraria se ha comido todo lo demás y que el Ministerio de Agricultura no ha desaparecido. También se dice que la ministra está defendiendo los intereses generales del Gobierno de España.

En el contexto proporcionado, se dice que la agricultura sostenible se refiere a la agricultura que es de acuerdo con los intereses generales del país, y que es más eficiente en el uso de recursos y en el desarrollo de la actividad agrícola. También se dice que la agricultura andaluza es sostenible y que se refiere a la agricultura que es de acuerdo con los intereses generales del país.

La agricultura sostenible es una forma de agricultura que se basa en el uso de métodos de producción que no dañen el medio ambiente. Es decir, se basa en la conservación de los recursos naturales y en el uso de técnicas de producción que no dejan dejar de producir los recursos naturales. La agricultura sostenible se considera una forma de agricultura que se basa en la conservación de los recursos naturales y en el uso de técnicas de producción que no dejan dejar de producir los recursos naturales.

Temperatura = 0.9

La agricultura sostenible es un tema de debate entre políticos y agricultores. Hay algunos que creen que la agricultura debería ser más sostenible y menos dependiente del uso de pesticidas y fertilizantes. Otros, sin embargo, creen que estos productos son esenciales para la producción de muchos cultivos y que el uso de menos de ellos no es viable. Aún hay otros que creen que la agricultura debería ser más sostenible y menos intensiva, y que esto se pueda conseguir con métodos de producción integrales, como la agricultura orgánica.

La agricultura sostenible es una forma de agricultura que tiene como objetivo el uso eficiente y sostenible de recursos naturales y humanos. La agricultura sostenible promueve el uso de métodos de agricultura eficiente que minimizan el uso de recursos y reduce la contaminación de los recursos naturales. También se basa en la diversificación de las actividades agrícolas y en el uso de métodos de conservación de la naturaleza para proteger los recursos naturales.

La agricultura sostenible es un concepto que se refiere a una forma de producción de alimento que tiene como objetivo ser sostenible y ser capaz de ser mantenida de manera duradera. La agricultura sostenible tiene como objetivo también proteger la vida y el medio ambiente, y dejar de perder recursos naturales. La agricultura sostenible puede ser llevada a cabo de manera integradora y colaborativa, que involucra a todos los actores del sector, de manera que se logre el mejor uso de los recursos disponibles y se proteja la sostenibilidad del sector.

Análisis

A partir de los resultados obtenidos se observa que la variabilidad entre ejecuciones es reducida (3 veces con cada parámetro) incluso al utilizar diferentes valores de temperatura. Este comportamiento es especialmente evidente en el contexto de este sistema RAG, donde la generación está fuertemente condicionada por la información recuperada.

En este sentido, los chunks proporcionados actúan como una restricción fuerte sobre el modelo generativo, limitando su capacidad de producir variaciones significativas en el contenido de las respuestas. Como consecuencia, incluso incrementando la temperatura, las diferencias observadas entre ejecuciones tienden a ser principalmente superficiales, afectando más a la formulación que al contenido semántico de las respuestas.

Este comportamiento sugiere que, en este escenario, la fase de recuperación tiene un mayor impacto en la salida final que los parámetros de decodificación del modelo, especialmente en preguntas de naturaleza factual o descriptiva.

En este trabajo se selecciona un rango de temperatura entre 0.5 y 0.7 para la fase de generación del modelo. Esta elección responde a la necesidad de encontrar un equilibrio entre coherencia y variabilidad en las respuestas generadas dentro del sistema RAG.

Valores inferiores de temperatura tienden a producir respuestas excesivamente deterministas y repetitivas, especialmente en combinación con contextos altamente estructurados, lo que puede limitar la expresividad del modelo. Por otro lado, valores superiores incrementan la diversidad de las respuestas, pero pueden introducir desviaciones respecto al contexto recuperado, afectando a la fidelidad de la información.

El rango seleccionado permite mantener una generación suficientemente estable para preservar la coherencia con el contexto, al mismo tiempo que introduce un grado moderado de variabilidad que favorece respuestas más naturales y menos repetitivas.

Adicionalmente, las preguntas del conjunto de evaluación han sido generadas utilizando una temperatura de 0.6, con el objetivo de asegurar una formulación equilibrada entre variabilidad lingüística y coherencia semántica en las consultas del dataset.

7.4. Comparación de respuestas: RAG frente a LLM

Para evaluar el impacto de la recuperación de información, se comparan las respuestas obtenidas mediante un modelo de lenguaje sin acceso a documentos externos (LLM) frente a las generadas por el sistema RAG desarrollado.

Ejemplo 1: "¿Quién es la señora Coves Botella?"**Respuesta LLM**

La señora Coves Botella es una profesora de matemáticas que trabaja en la Universidad de Barcelona.

Respuesta RAG

La señora Coves Botella es el presidente de la Legislatura en la sesión constitutiva del 3 de abril de 2008.

En este caso, el modelo sin recuperación genera una respuesta completamente inventada, atribuyendo a la persona una profesión y afiliación inexistentes dentro del corpus analizado. Por el contrario, el sistema RAG recupera documentos relevantes y responde utilizando la información contenida en ellos.

Ejemplo 2: "¿Quién es el señor Delgado Bonilla?"**Respuesta LLM**

El señor Delgado Bonilla es un personaje ficticio de la serie de televisión mexicana "La Casa de las Flores". Es interpretado por el actor Luis Gavina y es el esposo de Sofía Flores, una mujer de gran poder y influencia en la política mexicana.

Respuesta RAG

El señor Delgado Bonilla es un miembro del Partido Popular que intervino en una sesión del pleno de la Legislatura en la que se trató de proyectos de ley.

Nuevamente, el modelo de lenguaje produce una alucinación completa, generando una historia sin ninguna relación con el dominio parlamentario. La respuesta obtenida mediante RAG se fundamenta en la documentación recuperada y mantiene coherencia con el contexto legislativo.

Ejemplo 3: "¿Qué se habla el 29 de abril 2008 sobre el Consejo Asesor de RTVE en Andalucía?"**Respuesta RAG**

Se habla sobre el resultado de la votación para la designación de miembros del Consejo Asesor de RTVE en Andalucía, que se realizó en el 29 de abril de 2008.

Respuesta LLM

¿que se habla el 29 de abril 2008 sobre el Consejo Asesor de RTVE en Andalucía? El 29 de abril de 2008 se publicó una declaración del Consejo Asesor de RTVE en Andalucía sobre los recientes acontecimientos en la comunidad. La declaración fue publicada en el portal web de RTVE y se refirió a las acciones tomadas por el consejo asesor en el contexto de las protestas que se produjeron en Andalucía en contra de las reformas en el sistema audiovisual español. La declaración también se refirió a la decisión

del consejo asesor de RTVE en Andalucía de no emitir programas en catalán o en euskera en la región.

En esta consulta, el modelo sin recuperación vuelve a generar información ficticia que aparenta ser verosímil, pero que no está respaldada por ninguna evidencia documental. Por su parte, el sistema RAG proporciona una respuesta alineada con el contenido de las actas parlamentarias recuperadas, donde puede verificarse la respuesta en el archivo DSPA080004_5 chunk 0.

Análisis

Los resultados muestran una diferencia significativa entre ambos enfoques. El modelo de lenguaje utilizado de forma aislada carece de conocimiento específico sobre las sesiones parlamentarias almacenadas en el corpus y, ante preguntas sobre entidades o acontecimientos concretos, tiende a generar respuestas plausibles pero incorrectas, fenómeno conocido como *hallucination* o alucinación.

Por el contrario, el sistema RAG complementa la capacidad generativa del modelo con un mecanismo de recuperación documental basado en OpenSearch. Gracias a ello, las respuestas se apoyan en información realmente presente en las actas parlamentarias, reduciendo considerablemente la aparición de alucinaciones y aumentando la precisión factual de las respuestas. Estos ejemplos evidencian que, para dominios especializados y colecciones documentales cerradas, la incorporación de un componente de recuperación resulta fundamental para obtener respuestas fiables y trazables.

7.5. Experimentación con LLM-as-a-Judge

La evaluación de sistemas RAG no solo requiere analizar la capacidad del sistema para recuperar información relevante, sino también valorar la calidad de las respuestas generadas. Aunque las métricas de recuperación permiten medir el rendimiento de la fase de búsqueda documental, resultan insuficientes para evaluar aspectos como la precisión factual, la completitud o la coherencia de las respuestas. Por este motivo, en este trabajo se emplea una estrategia de evaluación basada en *LLM-as-a-Judge* [53, 13], donde un modelo de lenguaje actúa como evaluador automático de las respuestas producidas por el sistema.

Con el fin de analizar la posible influencia del modelo evaluador en los resultados obtenidos, se realiza una evaluación en dos etapas. En primer lugar, se utiliza el propio modelo *Mistral-7B-Instruct-v0.1* como juez para valorar las respuestas generadas. Posteriormente, los resultados se comparan con los obtenidos mediante un segundo modelo evaluador, seleccionado a partir de la configuración que presenta mejores métricas de recuperación. Esta comparación permite estudiar la existencia de posibles sesgos asociados

al evaluador y verificar la consistencia de las conclusiones extraídas sobre el rendimiento del sistema RAG.

7.5.1. Auto-evaluación con el modelo generador

En esta primera fase de la evaluación basada en LLM-as-a-Judge, se utiliza el propio modelo generativo del sistema (*Mistral-7B-Instruct-v0.1*) como evaluador de las respuestas producidas. Este enfoque, conocido como auto-evaluación, permite analizar la capacidad del propio modelo para juzgar la calidad de sus salidas en términos de relevancia, fidelidad al contexto y adecuación a la pregunta.

El número de fragmentos de contexto recuperados se fija en $k = 3$, valor seleccionado previamente como un compromiso óptimo entre calidad de respuesta y coste computacional, tal y como se analiza en secciones anteriores.

En concreto, se establece una longitud máxima de generación de 200 tokens (`max_new_tokens = 200`), una temperatura de 0.6, un valor de *top-p* de 0.9 y muestreo activado (`do_sample = True`). Estos parámetros se han seleccionado con el objetivo de equilibrar coherencia y variabilidad en las respuestas generadas, evitando tanto la excesiva determinismo como la generación de contenido incoherente.

Para la fase de evaluación, el mismo modelo generativo actúa como juez, recibiendo como entrada la pregunta, el contexto recuperado y la respuesta generada. A partir de esta información, debe asignar una puntuación entre 1 y 5 en tres dimensiones: relevancia del contexto, fidelidad de la respuesta al contexto (*faithfulness*) y relevancia de la respuesta respecto a la pregunta. Este enfoque permite evaluar de manera automática la calidad del sistema RAG sin necesidad de anotación manual.

Sistema	Context Relevance	Faithfulness	Answer Relevance
BM25	3.55	3.58	3.55
Embeddings	4.90	4.90	4.90
Híbrido OpenSearch	4.90	4.90	4.90
Híbrido RRF	4.65	4.62	4.67

Cuadro 7.4: Resultados de las métricas de generación.

Los resultados obtenidos mediante la evaluación automática con LLM-as-a-Judge muestran una tendencia general a otorgar puntuaciones elevadas de forma consistente en todas las configuraciones evaluadas, incluso en casos donde las métricas de retrieval indican una ausencia total de documentos relevantes recuperados.

Este comportamiento sugiere la presencia de un sesgo optimista en el modelo evaluador, que tiende a valorar positivamente la coherencia superficial y la fluidez de las respuestas generadas, sin penalizar adecuadamente la falta de fidelidad factual o la ausencia de evidencia en el contexto proporcionado.

En particular, se observa que respuestas generadas sin información relevante recuperada pueden recibir puntuaciones máximas en métricas como *faithfulness* o *answer relevance*, lo cual indica que el modelo juez no siempre realiza una verificación estricta de la consistencia entre la respuesta y el contexto.

Este fenómeno es una limitación conocida en enfoques basados en LLM-as-a-Judge, ya que el modelo evaluador puede priorizar la plausibilidad lingüística frente a la veracidad factual. Además, en el caso de la auto-evaluación, al emplearse el mismo modelo tanto para la generación como para la evaluación, se introduce un sesgo adicional derivado de la falta de independencia entre ambas tareas. Como consecuencia, los resultados obtenidos deben interpretarse como una aproximación relativa de calidad, más que como una medida absoluta del rendimiento del sistema.

Ejemplos:

¿Cuántas intervenciones ha realizado el dispositivo Infoca en los últimos incendios, y qué provincias han sido afectadas con mayor superficie?

Respuesta verdadera:

El dispositivo Infoca ha realizado un total de 12 intervenciones en los últimos incendios, con mayor superficie afectada en la provincia de Cádiz (985 hectáreas) y en la provincia de Almería (378 hectáreas).

Respuesta generada:

El dispositivo Infoca ha participado en 961 siniestros, de los cuales 327 fueron no forestales. En suelo forestal, se han producido 653 actuaciones, de las cuales 500 fueron conatos. Huelva ha sido la provincia con un mayor número de intervenciones, con un total de 211, seguida por Jaén y Almería. La superficie afectada por estos incendios es de 12.117

- hit: 0
- precision: 0.0
- recall: 0.0
- context relevance: 5
- faithfulness: 5
- answer relevance: 5

¿Cuáles son las principales demandas en el tema del empleo que se hacen en España, y cómo se relacionan con la gestión de la inmigración y la educación?

Respuesta verdadera:

”Las principales demandas en el tema del empleo que se hacen en España son la retirada del periodo de prueba y el rechazo de los contratos en origen,

ya que se consideran contratos de uso y tirar. Esto se relaciona con la gestión de la inmigración, ya que el inmigrante se convierte en una mercancía de usar y tirar. Además, los menores inmigrantes tienen dificultades para adaptarse en nuestras escuelas, lo que se relaciona con la educación.

Respuesta generada:

En España, las principales demandas en el tema del empleo se relacionan con la gestión de la inmigración y la educación en varios puntos. La primera preocupación es saber qué formación están recibiendo los inmigrantes y qué medidas está preparando la Consejería de Empleo, junto con otras Consejerías relacionadas, como la de Educación, para facilitar una formación adecuada que les permita introducirse en otros yacimientos de empleo.

- hit: 0
- precision: 0.0
- recall: 0.0
- context relevance: 5
- faithfulness: 5
- answer relevance: 5

7.5.2. Evaluación con un modelo juez externo

En esta segunda fase de la evaluación basada en LLM-as-a-Judge, se introduce un modelo evaluador externo con el objetivo de analizar la robustez de las métricas obtenidas en la auto-evaluación. En concreto, se emplea el modelo *Qwen2.5-7B-Instruct* [50] como juez independiente, con el fin de reducir posibles sesgos derivados del uso del mismo modelo tanto en la generación como en la evaluación de las respuestas.

El sistema de recuperación utilizado ha sido *Reciprocal Rank Fusion (RRF)*. Este método ha sido seleccionado ya que, en los experimentos previos de evaluación del sistema de recuperación, ha mostrado un rendimiento adecuado en términos de métricas de *Hit Rate*, *Precision* y *Recall* para $k = 3$, motivo por el cual se considera la configuración más adecuada para la evaluación de la fase generativa.

El número de *chunks* recuperados se fija en $k = 3$, valor previamente identificado como el mejor compromiso entre calidad de la información recuperada y coste computacional.

En cuanto al modelo generativo, se utiliza nuevamente *Mistral-7B-Instruct-v0.1* cuantizado a 4 bits, manteniendo la misma configuración de inferencia que en los experimentos anteriores: una longitud máxima de generación de 200 tokens (`max_new_tokens = 200`), temperatura de 0.6, valor de *top-p* de

0.9 y muestreo activado (`do_sample = True`). De este modo, se garantiza la comparabilidad de los resultados entre ambas fases de evaluación.

Para la fase de evaluación, se sustituye el modelo juez por *Qwen2.5-7B-Instruct* [50], que actúa como evaluador externo independiente. Este modelo recibe como entrada la pregunta, el contexto recuperado y la respuesta generada, y debe asignar puntuaciones entre 1 y 5 en tres dimensiones: relevancia del contexto, fidelidad de la respuesta al contexto (*faithfulness*) y relevancia de la respuesta respecto a la pregunta. Esta configuración permite analizar posibles sesgos del evaluador y obtener una estimación más robusta de la calidad del sistema RAG.

Sistema	Context Relevance	Faithfulness	Answer Relevance
Híbrido RRF	3.35	2.75	3.05

Cuadro 7.5: Resultados de las métricas de generación con juez externo.

Los resultados obtenidos muestran una mayor variabilidad en las puntuaciones en comparación con la auto-evaluación, situándose en rangos intermedios. Este comportamiento indica que el modelo juez externo realiza una evaluación más estricta de la fidelidad factual de las respuestas, penalizando aquellos casos en los que la información generada no está completamente respaldada por el contexto recuperado o presenta desviaciones respecto al contenido original.

En consecuencia, esta fase permite obtener una visión más realista del rendimiento del sistema RAG, reduciendo el sesgo optimista observado en la evaluación realizada con el propio modelo generador.

Ejemplos:

¿Qué son las dos reflexiones iniciales que la señora Pérez Fernández quiere hacer antes de entrar en materia?

Respuesta verdadera:

Las dos reflexiones iniciales que la señora Pérez Fernández quiere hacer antes de entrar en materia son: la primera es que estamos ante una nueva etapa, ante el comienzo de una nueva legislatura, y es el momento de fijar objetivos. La segunda es que cree que esta primera intervención está siendo también más moderada de lo que, habitualmente, está.

Respuesta generada:

La señora Pérez Fernández, en las dos reflexiones iniciales que quiere hacer, es decir, en el primer punto, quiere comentar sobre la situación del sector de la educación en España y la situación de los profesionales de la educación en el ámbito de la crisis económica, y en el segundo punto, quiere hablar sobre el impacto de la crisis económica en el sector de la educación y en los profesionales de la educación.

▪ hit: 0

- precision: 0.0
- recall: 0.0
- context relevance: 1
- faithfulness: 1
- answer relevance: 1

¿Cómo la propuesta del Partido Popular para garantizar más agua en Andalucía se relacionó con el trasvase del Ebro y cómo se benefició Málaga de esta acción?

Respuesta verdadera:

La propuesta del Partido Popular para garantizar más agua en Andalucía se relacionó con el trasvase del Ebro ya que el trasvase del Ebro suponía para Almería 95 hectómetros cúbicos, y la propuesta del Partido Popular garantizó 189 hectómetros cúbicos. Además, el trasvase del Ebro se olvidaba de la afectada en Andalucía, por la sequía, que era Málaga. Sin embargo, Málaga no se benefició de esta acción ya que no se beneficiaba ni en una sola gota.

Respuesta generada:

La propuesta del Partido Popular para garantizar más agua en Andalucía se relacionó con el trasvase del Ebro ya que se planteaba el trasvase de este río hacia Andalucía para garantizar una mayor cantidad de agua en la región. El trasvase del Ebro suponía para Almería 95 hectómetros cúbicos, y la propuesta del Partido Popular garantizó 189 hectómetros cúbicos para Almería. Más allá de esto, el trasvase del Ebro también se benefició a Málaga, ya que esta región no se beneficiaba en ninguna sola gota de agua del trasvase del Ebro.”

- hit: 1
- precision: 0.33
- recall: 0.33
- context relevance: 4
- faithfulness: 3
- answer relevance: 4

Capítulo 8

Conclusiones y trabajos futuros

El presente trabajo ha abordado el diseño, implementación y evaluación de un sistema de Recuperación Aumentada por Generación (RAG), analizando de forma sistemática el impacto de distintos componentes del proceso de recuperación y generación sobre la calidad de las respuestas producidas por un modelo de lenguaje. En un contexto donde los modelos fundacionales constituyen la base de numerosas aplicaciones de inteligencia artificial, las arquitecturas RAG se han consolidado como una alternativa eficaz para incorporar conocimiento externo y reducir las limitaciones derivadas del conocimiento paramétrico de los modelos.

La investigación ha permitido desarrollar una arquitectura RAG basada en OpenSearch, capaz de integrar distintos mecanismos de recuperación documental, incluyendo BM25, búsqueda vectorial mediante embeddings densos, recuperación híbrida nativa y un esquema híbrido basado en *Reciprocal Rank Fusion* (RRF). Sobre esta infraestructura se ha realizado una evaluación experimental empleando un corpus parlamentario especializado, caracterizado por un elevado grado de formalismo lingüístico, abundancia de entidades nominales y referencias legislativas específicas.

Los resultados obtenidos permiten extraer varias conclusiones relevantes. En primer lugar, el análisis del número de *chunks* recuperados muestra que la cantidad de contexto proporcionada al modelo generativo influye directamente en la completitud de las respuestas. A medida que aumenta el valor de k , el sistema incorpora una mayor cantidad de información contextual, permitiendo generar respuestas más detalladas y completas. Sin embargo, este beneficio presenta rendimientos decrecientes a partir de valores intermedios, observándose que incrementos adicionales del contexto no siempre se traducen en mejoras proporcionales de la calidad de las respuestas y pueden introducir información menos relevante.

Asimismo, se ha constatado que el número de *chunks* recuperados tiene

un impacto significativo sobre el coste computacional del sistema. El aumento del tamaño del contexto incrementa el número de tokens procesados durante la inferencia, provocando tiempos de generación considerablemente mayores. Los experimentos realizados muestran que valores intermedios de k , particularmente $k = 3$ y $k = 5$, ofrecen el mejor equilibrio entre calidad de respuesta y eficiencia computacional, siendo $k = 3$ una configuración especialmente atractiva cuando se prioriza el tiempo de respuesta.

En relación con la fase de recuperación, los resultados obtenidos mediante métricas clásicas de Information Retrieval indican que el sistema híbrido basado en *Reciprocal Rank Fusion* constituye la alternativa más robusta y consistente entre las evaluadas. Este enfoque obtiene los mejores resultados globales en términos de *Hit Rate*, *Precision* y *Recall*, confirmando que la combinación de señales léxicas y semánticas permite compensar las limitaciones inherentes de cada método por separado. BM25 muestra igualmente un comportamiento muy competitivo, especialmente en un dominio altamente dependiente de terminología específica y coincidencias léxicas exactas, mientras que los enfoques basados exclusivamente en embeddings densos presentan mayores dificultades para capturar ciertas particularidades del lenguaje parlamentario.

Por otra parte, los experimentos realizados sobre los parámetros de generación indican que la influencia de la temperatura resulta relativamente limitada dentro del contexto de un sistema RAG. La información recuperada actúa como una restricción fuerte sobre el proceso generativo, reduciendo significativamente la variabilidad de las respuestas incluso cuando se utilizan configuraciones más creativas. Este resultado pone de manifiesto que, en aplicaciones orientadas a preguntas factuales sobre colecciones documentales cerradas, la calidad de la recuperación tiene un impacto considerablemente mayor que los ajustes realizados sobre los parámetros de decodificación del modelo.

La comparación entre el modelo de lenguaje utilizado de forma aislada y el sistema RAG evidencia además la contribución fundamental del mecanismo de recuperación documental. Mientras que el modelo sin acceso al corpus tiende a generar respuestas plausibles pero incorrectas ante preguntas específicas del dominio, el sistema RAG es capaz de fundamentar sus respuestas en documentos reales recuperados del índice, reduciendo de manera significativa la aparición de alucinaciones y aumentando la trazabilidad de la información generada. Este resultado confirma la utilidad de las arquitecturas RAG para dominios especializados donde la precisión factual constituye un requisito esencial.

En cuanto a la evaluación automática mediante estrategias *LLM-as-a-Judge*, los resultados obtenidos revelan diferencias importantes entre la auto-evaluación y la evaluación realizada por un modelo independiente. La utilización del propio modelo generador como juez produce puntuaciones sistemáticamente elevadas, incluso en situaciones donde las métricas de recuperación

indican una ausencia total de información relevante. Este comportamiento evidencia un claro sesgo optimista asociado a la autoevaluación, en el que la coherencia lingüística superficial tiende a ser valorada por encima de la fidelidad factual.

Por el contrario, la evaluación realizada mediante un modelo juez externo proporciona puntuaciones más moderadas y consistentes con la calidad real de las respuestas observadas. Este comportamiento sugiere que la independencia entre el modelo generador y el evaluador resulta fundamental para obtener estimaciones más fiables del rendimiento de un sistema RAG. En consecuencia, los resultados obtenidos ponen de manifiesto las limitaciones de los enfoques de autoevaluación y refuerzan la necesidad de utilizar evaluadores independientes en futuros estudios.

En conjunto, los hallazgos obtenidos permiten concluir que el sistema híbrido basado en RRF combinado con un número moderado de *chunks* recuperados constituye la configuración más equilibrada dentro del sistema propuesto. Esta combinación logra un adecuado compromiso entre capacidad de recuperación, calidad de respuesta y coste computacional, proporcionando resultados consistentes en un dominio documental especializado.

Desde el punto de vista de los trabajos futuros, se identifican varias líneas de mejora relevantes. En primer lugar, sería conveniente ampliar el conjunto de evaluación mediante un mayor número de preguntas anotadas manualmente, lo que permitiría obtener estimaciones más robustas y estadísticamente representativas del comportamiento del sistema.

En segundo lugar, la incorporación de técnicas de *reranking* basadas en modelos *cross-encoder* podría mejorar la precisión de los documentos recuperados en las primeras posiciones del ranking, aumentando la densidad informativa del contexto proporcionado al modelo generativo y reduciendo la presencia de información irrelevante.

En tercer lugar, otra línea de trabajo de especial interés consiste en realizar un proceso de *fine-tuning* sobre los modelos empleados. Por una parte, el ajuste específico del modelo de *embeddings* utilizando un corpus parlamentario permitiría obtener representaciones vectoriales más adaptadas al vocabulario, las expresiones y las relaciones semánticas propias de este dominio, mejorando potencialmente el rendimiento de la recuperación densa y de los sistemas híbridos. Por otra parte, también resultaría interesante explorar el ajuste fino del propio modelo de lenguaje sobre este tipo de documentación especializada, con el objetivo de aumentar su capacidad para comprender el contexto legislativo, reducir la aparición de alucinaciones y generar respuestas más precisas y alineadas con el lenguaje parlamentario.

También se plantea como línea de trabajo futura la experimentación con modelos de lenguaje de mayor tamaño o con modelos especializados en tareas de razonamiento y evaluación, tanto para la generación de respuestas como para la fase de *LLM-as-a-Judge*. Esto permitiría estudiar con mayor profundidad el efecto de la capacidad del modelo sobre la fidelidad factual

y la calidad global del sistema.

Asimismo, resultaría de interés explorar mecanismos avanzados de selección dinámica del número de *chunks* recuperados, adaptando el valor de k a la complejidad de cada consulta. Este enfoque podría mejorar simultáneamente la eficiencia computacional y la calidad de las respuestas generadas.

Finalmente, otra línea de investigación prometedora consiste en complementar la evaluación automática con procesos de validación humana, combinando métricas automáticas y juicios expertos para obtener una valoración más fiable y representativa del rendimiento real del sistema en entornos de uso práctico.

En definitiva, este trabajo demuestra que las arquitecturas RAG constituyen una herramienta eficaz para mejorar la precisión y fiabilidad de los modelos de lenguaje en dominios especializados. Los resultados obtenidos confirman la importancia de una recuperación documental de calidad, muestran las ventajas de los enfoques híbridos basados en RRF y evidencian la necesidad de emplear estrategias de evaluación rigurosas para valorar adecuadamente el rendimiento de estos sistemas.

Bibliografía

- [1] AlbertBL. Información vectorial (embeddings) y su uso en bases de datos., 2024. <https://blog.albertosaenz.com/informacion-vectorial-embeddings-y-su-uso-en-bases-de-datos>.
- [2] Manoj Ghuhana Arivazhagan, Lan Liu, Peng Qi, Xinchu Chen, William Yang Wang, and Zhiheng Huang. Hybrid Hierarchical Retrieval for Open-Domain Question Answering. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 10680–10689. Association for Computational Linguistics, 2023.
- [3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv preprint arXiv:2310.11511*, 2023.
- [4] David Baum. *Generative AI and LLMs for Dummies: Snowflake Special Edition*. John Wiley & Sons, 2024. Snowflake Special Edition, PDF whitepaper.
- [5] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Neil C. Henderson, Eliza Rutherford, Katie Millican, Giovanni Agrawal, et al. Improving Language Models by Retrieving from Trillions of Tokens. *arXiv preprint arXiv:2112.04426*, 2021.
- [6] Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep Reinforcement Learning from Human Preferences. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [7] G. V. Cormack, C. L. A. Clarke, and Stefan Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM, 2009.
- [8] Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.

- [9] Tim Dettmers et al. bitsandbytes: 8-bit optimizers and matrix multiplication, 2026. Library for 8-bit/4-bit quantization and efficient LLM training/inference.
- [10] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019.
- [11] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise Zero-Shot Dense Retrieval without Relevance Labels. *arXiv preprint arXiv:2212.10496*, 2022.
- [12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [13] J. Gu, X. Jiang, Z. Shi, and et al. A survey on llm-as-a-judge. *The Innovation*, 7(6):101253, 2026.
- [14] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [15] Piotr Indyk and Rajeev Motwani. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1998.
- [16] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot Learning with Retrieval Augmented Language Models. *arXiv preprint arXiv:2208.03299*, 2022.
- [17] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- [18] Jeff Johnson, Matthijs Douze, and Herv   J  gou. Billion-scale similarity search with gpus. In *IEEE Transactions on Big Data*, 2019.
- [19] Karen Sp  rck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1):11–21, 1972.

- [20] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. Online manuscript, January 2025 draft.
- [21] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020.
- [22] Omar Khattab and Matei Zaharia. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48. Association for Computing Machinery, 2020.
- [23] Craig Larman and Victor R. Basili. *Iterative and Incremental Development: A Brief History*, volume 36. Computer, 2003.
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [25] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, Dense, and Attentional Representations for Text Retrieval. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2020.
- [26] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, volume 42, pages 824–836, 2018.
- [27] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *An Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [28] Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.
- [29] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient Estimation of Word Representations in Vector Space. In *Proceedings*

- of the International Conference on Learning Representations (ICLR) Workshop*, 2013.
- [30] Bonan Min, Hayley Ross, Elicor Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. Recent Advances in Natural Language Processing via Large Pre-trained Language Models: A Survey. *arXiv preprint arXiv:2111.01243*, 2021.
 - [31] Mina. Generación con recuperación aumentada (RAG), 2024. <https://blog.gopenai.com/retrieval-augmented-generation-rag-585aa903d6bd>.
 - [32] Rodrigo Nogueira and Kyunghyun Cho. Passage Re-ranking with BERT. *arXiv preprint arXiv:1901.04085*, 2019.
 - [33] NVIDIA Corporation. Cuda, 2026. GPU computing platform.
 - [34] NVIDIA Corporation. Nvidia, 2026. GPU hardware and CUDA platform provider.
 - [35] Fraidoon Omarzai. Word2Vec (CBOW, Skip-gram) en profundidad), 2024. <https://medium.com/@fraidoonomarzai99/word2vec-cbow-skip-gram-in-depth-88d9cc340a50>.
 - [36] OpenSearch Project. Opensearch, 2026. Open-source search and analytics engine.
 - [37] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
 - [38] Akshay Pachaar. Cross-encoders Vs Bi-encoders, 2025. https://www.linkedin.com/posts/akshay-pachaar_cross-encoders-vs-bi-encoders-information-activity-7255654210245951488-vNEV.
 - [39] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global Vectors for Word Representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543. Association for Computational Linguistics, 2014.
 - [40] Python Software Foundation. Elementtree, 2026. XML and HTML parsing library in Python standard library.

- [41] PyTorch Contributors. Pytorch, 2026. Deep learning framework.
- [42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. Technical report, OpenAI, 2019. Technical report.
- [43] Luis Angel Pérez Ramos. Modelos LLM y aplicaciones RAG paso a paso, 2024. <https://es.community.intersystems.com/post/modelos-llm-y-aplicaciones-rag-paso-paso-parte-ii-creando-el-contexto>.
- [44] Markku Räsänen. Uso de BM25 para optimizar la precisión de las preguntas y respuestas y las búsquedas, 2025. <https://confidentialmind.com/blog/okapi-bm25>.
- [45] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992. Association for Computational Linguistics, 2019.
- [46] Stephen Robertson and Karen Spärck Jones. Relevance Weighting of Search Terms. *Journal of the American Society for Information Science*, 27(3):129–146, 1976.
- [47] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [48] Luis Espinosa-Anke Shahul Es, Jithin James and Steven Schockaert. Ragas: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217*, 2023.
- [49] Ian Sommerville. *Software Engineering*. Pearson, 2016.
- [50] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [52] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report. *arXiv preprint arXiv:2402.05672*, 2024.
- [53] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, and et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 2023.